ANALYSIS AND OPTIMIZATION OF PARAMETERS OF WEAK DECOY STATE PROTOCOL FOR MITIGATION OF PHOTON NUMBER SPLITTING (PNS) ATTACKS ON THE BB84 QUANTUM KEY DISTRIBUTION ALGORITHM

A Report Submitted
in Partial Fulfilment for the Degree
of

BACHELOR OF TECHNOLOGY

in
Department of Computer Science and Engineering

by

Rishi Sarkar (Enrollment No. CSB22005)
Rahul Paul (Enrollment No. CSB22081)

Under the Supervision of

Prof. Nityananda Sarma
Professor
Department of Computer Science and Engineering, Tezpur University

School of Engineering
Department of Computer Science and Engineering
Tezpur University
Napaam – 784028, Assam, India

December 22, 2025

Department of Computer Science and Engineering
Tezpur University
Napaam – 784028, Assam, India

DECLARATION

We hereby declare that the work presented in this report entitled "Analysis and Optimization of parameters of Weak Decoy State Protocol for mitigation of Photon Number Splitting (PNS) Attacks on the BB84 Quantum Key Distribution Algorithm", is carried out by us. We have not submitted the matter embodied in this report for the award of any other degree or diploma of any other University or Institute. We have given due credit to the original authors/sources for all the words, ideas, diagrams, graphics, computer programs, experiments, results, that are not my original contribution. We have used quotation marks to identify verbatim sentences and given credit to the original authors/sources. We affirm that no portion of my work is plagiarized, and the experiments and results reported in the report are not manipulated. In the event of a complaint of plagiarism and the manipulation of the experiments and results, we shall be fully responsible and answerable.

Place: Napaam

Date: December 22, 2025

Name: Rishi Sarkar

Enrollment no: CSB22005

Signature:

Name: Rahul Paul

Enrollment no: CSB22081

Signature:

Department of Computer Science and Engineering
Tezpur University
Napaam – 784028, Assam, India

CERTIFICATE

This is to certify that the report entitled "Analysis and Optimization of parameters of Weak Decoy State Protocol for mitigation of Photon Number Splitting (PNS) Attacks on the BB84 Quantum Key Distribution Algorithm" submitted by **Rishi Sarkar (CSB22005)** and **Rahul Paul (CSB22081)** has been carried out under my supervision and guidance for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering during the session 2022 – 2026 at Tezpur University. To the best of my knowledge, the matter embodied in the report has not been submitted to any other university/institute for the award of any Degree or Diploma.

(Signature)

Nityananda Sarma
Professor
Dept. of CSE, Tezpur University

Date: December 22, 2025

Department of Computer Science and Engineering
Tezpur University
Napaam – 784028, Assam, India

CERTIFICATE by HoD

This is to certify that the report entitled "Analysis and Optimization of parameters of Weak Decoy State Protocol for mitigation of Photon Number Splitting (PNS) Attacks on the BB84 Quantum Key Distribution Algorithm" is submitted by **Rishi Sarkar (CSB22005)** and **Rahul Paul (CSB22081)**. They have completed their project work successfully as needed for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering during the session 2022-2026 at Tezpur University. To the best of my knowledge, the matter embodied in the dissertation has not been submitted to any other university/institute for the award of any Degree or Diploma.

(Signature)

Head of the Department
Department of Computer Sc & Engineering
Tezpur University

Date: December 22, 2025

Department of Computer Science and Engineering
Tezpur University
Napaam – 784028, Assam, India

Certificate by the Examiner

This is to certify that the report entitled "Analysis and Optimization of parameters of Weak Decoy State Protocol for mitigation of Photon Number Splitting (PNS) Attacks on the BB84 Quantum Key Distribution Algorithm" submitted by **Rishi Sarkar (CSB22005)** and **Rahul Paul (CSB22081)** in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering during the session 2022-2026 at Tezpur University has been examined by me and is found satisfactory for the award of the degree.

This approval does not necessarily endorse or accept every statement made, opinion expressed, or conclusion drawn as recorded in the report. It only signifies the acceptance of this report for the purpose for which it is submitted.

(Examiner)

Place:

Date:

ACKNOWLEDGEMENTS

We take this opportunity to extend our heartfelt gratitude to our supervisor, **Prof. Nityananda Sarma**, Professor, for giving us the opportunity to carry out our work under his guidance and providing us with ample support through the course of the project.

We would also like to thank the Head of the Department, Dept. of Computer Science and Engineering, and all the faculty members and Staff of the department at Tezpur University for their valuable guidance and cooperation throughout the project.

Our sincere thanks and appreciation go to our friends and family members who have directly or indirectly helped us out with their abilities.

Rishi Sarkar (CSB22005)

Rahul Paul (CSB22081)

ABSTRACT

The advent of quantum computing poses a significant threat to classical cryptographic systems, necessitating the development of quantum-resistant security protocols. Quantum Key Distribution (QKD), particularly the BB84 protocol, offers a solution with security guaranteed by the laws of physics. However, the theoretical security of BB84 relies on idealized hardware, a condition not met in practice. This report presents a detailed technical methodology addressing two critical aspects of practical QKD: intrinsic channel noise and source-based vulnerabilities.

Initially, a sophisticated simulation framework was constructed using Python and the Qiskit-Aer library to model the impact of realistic quantum channel noise, featuring advanced models such as Amplitude Damping, Phase Damping, and Depolarizing errors. This analysis quantifies the performance degradation due to distance and noise and confirms that while environmental factors systematically increase the baseline Quantum Bit Error Rate (QBER), the protocol's ability to detect an intercept-resend attack remains robust and statistically distinct.

Building on this foundation, the project tackles the more severe vulnerability of the Photon-Number-Splitting (PNS) attack, which arises from the use of attenuated lasers in real-world implementations. To counter this, we developed a second, high-fidelity numerical simulation focused on the decoy state method. This model serves as the fitness function for a numerical optimization algorithm, which systematically searches the parameter space—including the intensities and probabilities of signal and decoy states—to determine the optimal protocol configuration that maximizes the secure key generation rate for any given scenario.

Our findings reveal that the optimal number of decoy states is not static but is a dynamic choice dependent on the interplay between channel loss and the statistical limitations of a finite block size. We produce a "strategy map" that identifies the zones of optimality for different decoy state protocols (2-state, 3-state, and 4-state), demonstrating the trade-offs between protocol complexity and statistical cost. In summary, this report provides a comprehensive, quantitative validation of BB84's security in non-ideal conditions, from fundamental noise analysis to advanced protocol optimization, offering clear guidelines for the challenges inherent in its real-world deployment.

CONTENTS

List of Symbols and Abbreviations **xii**

1 PROJECT INTRODUCTION AND OBJECTIVES **1**

1.1 Problem Statement 1

1.2 Core Objectives 1

1.3 Scope and Delimitations 2

2 LITERATURE REVIEW **3**

2.1 Foundational Principles of Quantum Cryptography 3

2.2 Information-Theoretic Security Proofs 3

2.3 Modeling the Real-World Quantum Channel and Its Vulnerabilities 4

2.4 The Decoy State Method: A Practical Solution 4

2.5 Classical Post-Processing 5

2.6 Positioning of the Current Research 5

3 THEORETICAL FRAMEWORK: MODELING THE NOISY QUANTUM CHANNEL **6**

3.1 The Ideal vs. The Real Quantum Channel 6

3.2 Amplitude Damping: The Physics of Photon Loss 6

3.2.1 Mathematical Model: The Beer-Lambert Law 6

3.2.2 Engineering Formulation: Attenuation in Decibels (dB) 7

3.2.3 Implementation in the Simulation 7

3.3 Phase Damping: The Loss of Quantum Coherence 7

3.3.1 Physical Mechanism: Dephasing 7

3.3.2 Impact on the BB84 Protocol 8

3.3.3 Implementation in the Simulation 8

3.4 Depolarizing Error: A Generic Noise Model 8

3.4.1 Physical Mechanism: Entropic Scrambling 8

3.4.2 Implementation in the Simulation 8

3.5 The Combined Noise Model: A Realistic Channel Approximation 8

4 THEORETICAL FRAMEWORK: INFORMATION SECURITY AND PROTO-

COL METRICS **10**
4.1 Introduction: The Problem with the Sifted Key 10
4.2 Part 1: Error Correction (also known as Reconciliation) 10
4.2.1 How Parity-Based Error Correction Works: An Analogy 10
4.3 Part 2: Privacy Amplification 12
4.3.1 How Privacy Amplification Works: The “Data Blender” Analogy . . . 12
4.4 The Quantum Bit Error Rate (QBER): A Dual-Purpose Metric 13
4.5 The Security Threshold: A Deliberate Balance 13
4.6 Quantifying Information Leakage: Privacy Amplification and Shannon Entropy 14

5 SIMULATION METHODOLOGY AND IMPLEMENTATION **15**
5.1 Simulation Environment and Libraries 15
5.2 Architectural Overview 15
5.3 The Noise Model Engine (create_noise_model) 16
5.4 The Simulation Orchestrator (run_experiment) 16
5.5 Data Aggregation and Surface Fitting 17

6 RESULTS: IN-DEPTH ANALYSIS AND INTERPRETATION **18**
6.1 Analysis of 3D QBER Surfaces (Individual Noise Models) 19
6.1.1 Observations 19
6.1.2 Interpretation 21
6.2 Analysis of the 3D QBER Surface (Combined “Real-World” Model) 21
6.2.1 Observations 21
6.2.2 Interpretation 23
6.3 Analysis of Secure Key Rate vs. Distance 24
6.3.1 Observations 24
6.3.2 Interpretation 24
6.4 Analysis of 2D QBER Slices from the Combined Model 25
6.4.1 QBER vs. Channel Noise at a Fixed Distance 25

7 DISCUSSION **26**
7.1 Synthesis of Primary Findings 26
7.2 Limitations of the Simulation Model 26

8 THE PNS VULNERABILITY: FROM IDEAL TO REAL-WORLD QKD **28**
8.1 The Imperfection of Real-World Sources 28
8.2 Eve’s Attack Strategy: The Photon-Number-Splitting (PNS) Attack 28

9 THE DECOY STATE COUNTERMEASURE **30**
9.1 Introducing Decoy States 30

9.2 The Protocol in Action 30
9.3 The Statistical ”Smoking Gun”: How Eve is Caught 31
9.4 Key Distillation and Security Guarantee 31

10 SIMULATION AND OPTIMIZATION FRAMEWORK 33
10.1 Simulation Environment 33
10.2 Physical Model Parameters 33
10.3 The Core Simulation Engine 34
10.4 The Need for an Optimization Layer 34

11 RESULTS AND DISCUSSION 35
11.1 Preliminary Analysis with Fixed Parameters 35
11.1.1 Observations 36
11.1.2 Initial Interpretation 36
11.2 Robust Analysis with Full Parameter Optimization 37
11.2.1 Observations 37
11.2.2 Interpretation: The Zones of Optimality 38
11.3 Deviation and Final Inferences 38

12 FINALCONCLUSION AND AVENUES FOR FUTURE RESEARCH 40
12.1 Conclusion 40
12.2 Avenues for Future Research 40
12.2.1 Incremental Enhancements to the Simulation Model 40
12.2.2 Primary Future Direction: The Quantum Repeater Placement Problem . 41

A Source Code 43

REFERENCES 66

B Plagiarism Report 67
B.1 Plagiarism Report 68

LIST OF FIGURES

6.1 QBER Surface for Depolarizing Noise Model, showing the clear separation between the attacked (upper) and unattacked (lower) channels. 19
6.2 QBER Surface for Amplitude Damping Noise Model. The behavior is qualitatively similar to the depolarizing model. 20
6.3 QBER Surface for the Combined Noise Model (View 1). 21
6.4 QBER Surface for the Combined Noise Model (View 2), highlighting the gradient along the distance axis. 22
6.5 QBER Surface for the Combined Noise Model (View 3), highlighting the gradient along the noise rate axis. 23
6.6 Secure Key Rate (SKR) as a Function of Distance for different levels of intrinsic channel noise. 24
6.7 QBER vs. Intrinsic Noise Rate at a Fixed Distance of 30 km. 25

11.1 Optimal number of decoy states determined using fixed, non-optimized protocol parameters. The result is a simple plateau. 36
11.2 Optimal number of decoy states determined using a full optimization of protocol parameters at each data point. This reveals a complex, multi-zoned ”strategy map.” 37

LIST OF SYMBOLS AND ABBREVIATIONS

QKD	Quantum Key Distribution
PNS	Photon Number Splitting
QBER	Quantum Bit Error Rate
GLLP	Gottesman-Lo-Lütkenhaus-Preskill (security proof)
VOA	Variable Optical Attenuator
S	Secure Key Generation Rate
μ, ν	Average photon number of a coherent state
Y_n	Yield of an n-photon state
e_n	Error rate of an n-photon state
Q_μ	Overall gain for a state with average photon number μ
E_μ	Overall QBER for a state with average photon number μ
$H_2(x)$	Binary Shannon Entropy Function

CHAPTER 1

PROJECT INTRODUCTION AND OBJECTIVES

1.1 Problem Statement

While the theoretical framework of the BB84 protocol guarantees unconditional security, its practical implementation is invariably subject to a hierarchy of imperfections. At a fundamental level, the quantum channel itself is not ideal; interactions with the environment manifest as noise, which degrades the quantum signal and introduces errors into the final key. Critically, these naturally occurring errors can provide cover for an eavesdropper, potentially masking their presence. Therefore, a thorough understanding and quantitative modeling of this baseline channel noise is the first essential step in defining the operational limits and verifying the security of any real-world QKD system.

Beyond general channel noise, a more severe and targeted vulnerability arises from non-ideal hardware. The theoretical assumption of a perfect single-photon source is not met in practice, where attenuated lasers are used. This gives rise to the critical **Photon Number Splitting (PNS) attack**, which allows an eavesdropper to gain information without introducing detectable errors, fundamentally undermining the protocol's security.

Although a powerful countermeasure known as the **weak decoy state protocol** has been developed to mitigate the PNS attack, its effectiveness is not absolute. The theoretical security of the decoy method itself relies on a set of protocol parameters and assumptions about the channel that may not hold perfectly in a practical scenario.

This project, therefore, addresses this multi-layered problem. We first establish a robust model of a realistic, noisy quantum channel. We then use this foundation to investigate the PNS attack and analyze the practicality and performance of the decoy state protocol, with the ultimate goal of optimizing its parameters to maximize the secure key rate under real-world conditions.

1.2 Core Objectives

The central goal of this project is to conduct a comprehensive, simulation-based investigation that bridges the gap between the idealized theory of BB84 and the challenges of its practical deployment. The specific objectives are structured to build from foundational analysis to ad-

vanced optimization:

1. **Develop Physically-Grounded Noise Models:** To implement mathematical models of common quantum channel noise—including Amplitude Damping, Phase Damping, and Depolarizing errors—based on their underlying physical mechanisms.

2. **Quantify the Impact of Distance and Validate Eavesdropping Detection:** To model distance-based photon loss and rigorously test if the QBER signature of a basic intercept-resend attack remains statistically distinguishable from the baseline QBER of a noisy, long-distance channel.

3. **Analyze Protocol Efficiency under Noise:** To determine the Secure Key Rate (SKR) as a function of both noise and distance, thereby mapping the fundamental performance envelope within which the ideal protocol is viable.

4. **Investigate Advanced Vulnerabilities:** To understand the theoretical limitations of BB84's security proof that lead to practical attacks, with a primary focus on the Photon Number Splitting (PNS) attack.

5. **Simulate the Decoy State Countermeasure:** To implement the weak decoy state protocol, demonstrating how it counters the PNS attack by allowing for the experimental estimation of single-photon statistics, thus forcing an attacker to create detectable anomalies.

6. **Optimize Decoy State Protocol Parameters:** To develop a numerical optimization framework that analyzes the practicality of the decoy state method. This involves finding the optimal set of parameters (e.g., number of decoys, their intensities, and probabilities) that maximizes the SKR across varying distances and block sizes, thereby comparing the protocol's practical performance against its theoretical backing.

1.3 Scope and Delimitations

This investigation focuses on the quantum communication and security analysis phases of the BB84 protocol and its decoy state variant.

- **Included:** Quantum state preparation; modeling of the quantum channel with both generic noise and photon-number statistics; simulation of both intercept-resend and PNS attacks; measurement; sifting; QBER calculation; and calculation of the secure key rate based on the GLLP framework and decoy state analysis.

- **Excluded (Conceptually Discussed):** The full algorithmic implementation of classical post-processing (Error Correction and Privacy Amplification). While the *information cost* of these steps is a critical part of our secure key rate calculation (e.g., using the Shannon entropy and inefficiency factors), the specific protocols like Cascade are not simulated.

CHAPTER 2

LITERATURE REVIEW

The field of Quantum Key Distribution (QKD) rests upon a rich history of theoretical and experimental advancements that have established its viability as a secure communication technology. This review surveys the seminal and contemporary works that form the foundation for the protocol's principles, its practical vulnerabilities, and the sophisticated countermeasures that enable its real-world application.

2.1 Foundational Principles of Quantum Cryptography

The conceptual genesis of using quantum mechanics for security can be traced to the work of Stephen Wiesner, who, in the late 1960s, proposed the idea of "conjugate coding" to create unforgeable quantum money [13]. While unpublished for over a decade, his work introduced the revolutionary concept that information could be encoded in non-orthogonal quantum states (e.g., rectilinear vs. diagonal polarization), and that any attempt to measure one set of states would irrevocably destroy the information encoded in the other.

This principle was brilliantly adapted for key distribution by Charles H. Bennett and Gilles Brassard in their seminal 1984 paper, which introduced the protocol now famously known as BB84 [1]. This paper laid the complete groundwork for the protocol, detailing the process of encoding information onto qubits, transmitting them, and using two different measurement bases to establish a shared secret. Crucially, it was the first to articulate that the disturbance induced by an eavesdropper's measurement on a quantum channel could be used to detect their presence, thereby guaranteeing the security of the sifted key. An alternative, entanglement-based approach was later proposed by Artur Ekert [4] with the E91 protocol, which, while different in implementation, relied on the same fundamental quantum principles.

2.2 Information-Theoretic Security Proofs

The original BB84 paper provided a strong intuitive argument for security, but a rigorous, mathematical proof of unconditional security took many more years to develop. The modern understanding of QKD security rests on information-theoretic proofs that grant the eavesdropper unlimited computational power and allow for the most general attacks permitted by quantum mechanics.

A pivotal breakthrough in this area was the Shor-Preskill security proof [12]. This elegant work established a powerful connection between the security of the BB84 protocol and the mathematics of quantum error-correcting codes. It proved that if the observed error rate is below a certain threshold, a secure key can be guaranteed. A key result, central to this project's analysis, is that the information Eve can gain about the final key is bounded by the error rate she induces. This work provided the first truly robust proof for the **≈11% QBER threshold** against general attacks, a cornerstone of our simulation's security analysis. The theoretical framework for privacy amplification itself predates QKD, originating in classical information theory with the work of Csiszár and Körner [3] on broadcast channels with confidential messages. Modern security proofs, such as those detailed in the comprehensive review by Renner [11], use advanced mathematical formalisms to provide even tighter bounds on the final key length.

2.3 Modeling the Real-World Quantum Channel and Its Vulnerabilities

The foundational security proofs rested on a critical idealization: a perfect, lossless, and noise-free quantum channel. Simulating QKD realistically requires a deep understanding of the decoherence processes that affect qubits in transit. The dominant loss mechanism in fiber-optic systems is photon absorption, a phenomenon accurately described by the Beer-Lambert Law, which predicts an exponential decay of signal intensity with distance [8].

Broader noise processes, such as dephasing (phase damping) and entropic scrambling (depolarizing), are formally modeled using the operator-sum representation for quantum operations, as detailed in standard texts like Nielsen and Chuang [10]. Experimental work, such as the review by Gisin et al. [5], has consistently confirmed that these models accurately reflect the primary sources of error in real-world QKD links.

However, the most critical real-world imperfection is not in the channel but in the source. The security proofs assume perfect single-photon sources, which are technologically demanding. Practical QKD systems instead use highly attenuated lasers, which sometimes emit multi-photon pulses. This opens a critical vulnerability known as the **Photon-Number-Splitting (PNS) attack**. In a PNS attack, an eavesdropper can steal one photon from a multi-photon pulse without causing a detectable error. The comprehensive security analysis by Gottesman, Lo, Lütkenhaus, and Preskill (GLLP) [6] formally quantified this threat, showing that without a solution, the secure key rate and maximum distance of practical systems were severely limited.

2.4 The Decoy State Method: A Practical Solution

The severe limitations imposed by the PNS attack were effectively neutralized by the **decoy state method**, a landmark innovation that made secure, long-distance QKD with imperfect sources practical. The core idea, first introduced by Hwang [7], is for Alice to randomly vary the intensity of her laser pulses. These "decoy" states of different intensities set a statistical trap

for Eve, allowing the detection of PNS attacks.

The method was made rigorous and practical by subsequent works, which provided the complete mathematical machinery. The work by Ma, Qi, Zhao, and Lo [9], along with others, provided the robust framework showing that only a few decoy states (e.g., one weak decoy and one vacuum state) are sufficient to tightly bound Eve's information. This allows Alice and Bob to calculate a secure key rate based on experimentally verifiable parameters, effectively solving the PNS problem.

2.5 Classical Post-Processing

The final, and often most complex, part of a QKD system is the classical post-processing. The process of error correction (reconciliation) must be done with minimal information leakage. The **Cascade protocol**, proposed by Brassard and Salvail [2], was one of the first influential interactive protocols designed for this purpose, using parity checks to efficiently find and correct errors. The information leaked during this step, combined with the information gained by Eve from the channel, determines the amount of compression required in the final privacy amplification step.

2.6 Positioning of the Current Research

Our research builds directly upon this established body of work. We first validate the principles of the Shor-Preskill proof by simulating the BB84 protocol in a realistic, noisy channel. We then address the PNS attack by implementing and analyzing the decoy state method. While this method provides a secure framework, its performance is a complex function of the chosen protocol parameters. This project aims to move beyond the proof-of-concept and conduct a detailed analysis of the protocol's practicality by simulating and optimizing the performance of the decoy state BB84 protocol under a wide range of realistic conditions.

CHAPTER 3

THEORETICAL FRAMEWORK: MODELING THE NOISY QUANTUM CHANNEL

3.1 The Ideal vs. The Real Quantum Channel

In an ideal world, a qubit transmitted by Alice would arrive at Bob's location in the exact state it was prepared. The real world, however, is an open quantum system where the qubit (the system) inevitably interacts with its surrounding environment. These uncontrolled interactions lead to a degradation of the quantum state, a process known as decoherence. Our simulation models this decoherence through specific, well-understood noise channels.

3.2 Amplitude Damping: The Physics of Photon Loss

Amplitude Damping is the quantum mechanical process that describes the loss of energy from a quantum system to its environment. For QKD systems using photons as qubits, this is the most dominant source of error, representing two main physical phenomena:

1. **Spontaneous Emission:** If using an atom-like system, an excited state ($|1\rangle$) can spontaneously decay to the ground state ($|0\rangle$), releasing energy.

2. **Photon Absorption:** In fiber-optic QKD, the photon qubit can be absorbed by impurities or imperfections within the glass fiber, causing it to be lost entirely. The simulation treats a lost photon as an error, as Bob's detector fails to register an event.

3.2.1 Mathematical Model: The Beer-Lambert Law

The probability of a photon being absorbed as it travels through a medium is described by the Beer-Lambert Law. The law states that the intensity of light decreases exponentially as it passes through a material. The fundamental differential equation is $dI/dx = -\alpha I$, meaning the change in intensity I over a small distance dx is proportional to the current intensity and an absorption coefficient α.

Integrating this gives the familiar form:

$$I(x) = I_0 \cdot e^{-\alpha x} \tag{3.1}$$

Where:

- $I(x)$ is the intensity of light after traveling a distance x.
- I_0 is the initial intensity.
- α is the attenuation coefficient of the medium (our optical fiber).
- x is the distance.

Since the probability of a single photon's transmission is analogous to the intensity ratio $I(x)/I_0$, the survival probability is $P(\text{survival}) = e^{-\alpha x}$.

3.2.2 Engineering Formulation: Attenuation in Decibels (dB)

In telecommunications, it is standard practice to measure loss in a logarithmic unit called the decibel (dB), as it turns exponential decay into linear addition. The loss in dB is defined as:

$$\text{Loss(dB)} = -10 \cdot \log_{10}(I(x)/I_0) \tag{3.2}$$

A standard, high-quality telecommunication optical fiber has a typical loss of 0.2 dB/km at the 1550 nm wavelength commonly used for QKD. This linear loss factor makes calculations simple. For a fiber of length L (in km), the total loss is:

$$\text{Total Loss(dB)} = 0.2 \cdot L \tag{3.3}$$

To convert this back to a survival probability for our simulation, we rearrange the formula:

$$P(\text{survival}) = I(x)/I_0 = 10^{(-\text{Total Loss(dB)}/10)} \tag{3.4}$$

3.2.3 Implementation in the Simulation

The simulation uses this engineering formula directly. For a given distance d, it first calculates the total loss in dB, then converts this to a survival probability. The probability of an amplitude damping error (i.e., the photon being lost) is then $P(\text{error}) = 1 - P(\text{survival})$. This value is used to parameterize Qiskit's `amplitude_damping_error` function, providing a physically realistic model of distance-based signal loss.

3.3 Phase Damping: The Loss of Quantum Coherence

3.3.1 Physical Mechanism: Dephasing

Phase Damping, or dephasing, is a subtle but critical noise process. It describes the loss of phase information in a quantum superposition without any loss of energy. It arises from random

fluctuations in the qubit's environment that affect its energy levels, causing the relative phase between the $|0\rangle$ and $|1\rangle$ components of its wavefunction to randomize. It's akin to a spinning top (a superposition state) wobbling on a vibrating floor; it doesn't stop spinning (lose energy), but its axis of rotation gets scrambled.

3.3.2 Impact on the BB84 Protocol

This noise type has a very specific effect on BB84. The Z-basis states ($|0\rangle$ and $|1\rangle$) are not superpositions and have no relative phase to destroy. Therefore, they are largely unaffected by phase damping. However, the X-basis states ($|+\rangle$ and $|-\rangle$) are perfect superpositions whose properties depend entirely on a precise phase relationship. Phase damping destroys this relationship, effectively collapsing the qubit into either a $|0\rangle$ or $|1\rangle$ state. If Bob then measures this collapsed state in the X-basis, his result will be random, directly introducing errors into the sifted key for all qubits transmitted in the X-basis.

3.3.3 Implementation in the Simulation

This is modeled using Qiskit's `phase_damping_error` function, which is applied to all quantum gates in the simulation. This correctly captures the effect of dephasing on the qubits that undergo a Hadamard (H) gate transformation (i.e., those in the X-basis).

3.4 Depolarizing Error: A Generic Noise Model

3.4.1 Physical Mechanism: Entropic Scrambling

Depolarizing noise is not tied to a single physical process but serves as a useful generic model for a channel that randomly scrambles the qubit's state. It assumes that with some probability p, the qubit's state is discarded and replaced with the maximally mixed state (pure random noise). This is equivalent to applying one of the three Pauli errors (X: bit-flip, Y: bit- and phase-flip, Z: phase-flip) with equal probability.

3.4.2 Implementation in the Simulation

This is implemented using Qiskit's `depolarizing_error` function. It provides a baseline error level that affects all qubit states, regardless of their basis, simulating a variety of small, unspecific interactions.

3.5 The Combined Noise Model: A Realistic Channel Approximation

To simulate a real-world environment, a combined noise model was created. This model assumes that all three noise processes occur simultaneously. In the simulation, the total `error_rate` is divided among the three noise types, and the resulting error channels are composed into

a single, more complex quantum error. This is applied on top of the always-present, distance-dependent amplitude damping, providing the most realistic channel approximation in this project.

CHAPTER 4

THEORETICAL FRAMEWORK: INFORMATION SECURITY AND PROTOCOL METRICS

4.1 Introduction: The Problem with the Sifted Key

After Alice and Bob perform the quantum part of BB84 and "sift" their results by comparing bases, they are left with a shared string of bits called the **Sifted Key**. This Sifted Key has two major problems that make it completely unusable for cryptography:

1. **It contains errors.** Due to noise in the quantum channel and imperfections in the hardware, Bob's Sifted Key is not a perfect copy of Alice's. It's just *mostly* the same. Cryptography demands perfectly identical keys.

2. **It is not perfectly secret.** An eavesdropper, Eve, has some partial information about the key. She might not know the whole key, but she might have a good guess about certain bits. Furthermore, the process of fixing the errors will leak *even more* information to her.

To solve these two problems, Alice and Bob must perform a two-step, purely classical (non-quantum) process on their Sifted Keys over a public, authenticated channel.

4.2 Part 1: Error Correction (also known as Reconciliation)

The Goal: To find and fix all the errors in the Sifted Key so that Alice and Bob have perfectly identical bit strings.
The Challenge: How can they find the differences in their keys without simply reading them out loud to each other? If they did that, Eve would learn the entire key, and the whole QKD process would be pointless.
The Solution: They use an interactive protocol that reveals information *about* their key, but not the key itself, to help them pinpoint the errors. The most famous protocol is called **Cascade**, but a simpler one to understand is based on **parity checks**.

4.2.1 How Parity-Based Error Correction Works: An Analogy

Imagine Alice and Bob are in separate rooms and have each written down the same long list of 100 random numbers (their Sifted Key). They suspect a few numbers on Bob's list are wrong.

They can talk over an intercom (the public channel) that they know Eve is listening to.

1. **Step 1: Block and Compare Parity**
   - Alice and Bob group their lists into smaller blocks (e.g., 10 blocks of 10 numbers each).
   - For the first block, Alice calculates the **parity**. This is a simple check: is the sum of the numbers in her block even or odd?
   - Alice announces over the intercom: "The sum of my first block is **even**."
   - Bob calculates the sum of his first block. If his sum is also even, they agree this block is likely correct and move on.
   - However, if Bob's sum is **odd**, they immediately know there is at least one error *somewhere* in that block. Eve also hears this, but she only learns that an error exists; she doesn't learn any of the numbers.
2. **Step 2: Pinpoint the Error with Binary Search**
   - Now that they've identified a faulty block, they need to find the specific error. They use a "divide and conquer" strategy.
   - Alice says: "Okay, for that bad block, let's check the parity of the **first half** (the first 5 numbers)." She announces the result (e.g., "odd").
   - Bob checks the parity of his first half. If his is also odd, they know the error must be in the *second half* of the block. If his is even, they know the error is in the *first half*.
   - They repeat this process, dividing the suspicious section in half each time, until they have narrowed it down to a single, incorrect number. Bob then flips that bit.
3. **Step 3: Shuffle and Repeat**
   - A single error can be found this way. But what if a block has two errors? An even number of errors won't be caught by a simple parity check (e.g., two flipped bits might still result in an "even" sum).
   - To solve this, after one full pass, Alice and Bob randomly shuffle the order of their key bits (using the same random sequence) and repeat the entire process of blocking and comparing parities. They do this multiple times. The random shuffling ensures that two errors that were in the same block in the first pass will likely be in different blocks in later passes, where they can be detected and corrected.

The Outcome: After several passes, Alice and Bob can be extremely confident that their keys are now identical.

The Cost of Error Correction: This process requires a lot of public communication. Eve listens to this entire conversation. While she doesn't learn the key itself, she learns the parity of many different subsets of the key. Each announcement gives her a little bit more information, reducing the key's overall secrecy. This leads directly to the need for the second step.

4.3 Part 2: Privacy Amplification

The Goal: To take the now-identical key and distill from it a shorter, but perfectly secret, final key. This step is designed to destroy any and all partial information that Eve possesses.
The Challenge: How can you eliminate information that you don't even know Eve has? You have to assume the worst: that Eve has collected the maximum possible information allowed by the QBER and the error correction discussion.
The Solution: Alice and Bob use a special one-way mathematical function called a **cryptographic hash function**.

4.3.1 How Privacy Amplification Works: The "Data Blender" Analogy

Think of a hash function as an irreversible, industrial-strength data blender. It has two key properties:

1. **It Shrinks Data:** It takes a long input (your corrected key) and produces a much shorter, fixed-length output (the final secure key).

2. **The Avalanche Effect:** It is extremely sensitive. Changing even a *single bit* of the input will cause the output to change completely and unpredictably.

Here's the process:

1. **Step 1: Agree on a Blender**

    - Alice and Bob publicly agree on which standard hash function to use (e.g., SHA-256). This is not a secret.

2. **Step 2: Calculate the Amount to Shrink**

    - Based on the initial QBER they measured and how much information they estimate was leaked during error correction, they calculate exactly how much partial information Eve *could* have. This tells them how much they need to shrink their key to make her information useless.
    - Let's say their corrected key is 1000 bits long, and they calculate that Eve might have 300 bits worth of information about it. They decide to hash their 1000-bit key down to a final key of 700 bits (1000 - 300).

3. **Step 3: Blend the Key**

- Alice takes her 1000-bit corrected key and runs it through the SHA-256 function, keeping only the first 700 bits of the output.
- Bob does the exact same thing with his identical 1000-bit key.
- Because their input keys were identical, their output "blended" keys will also be perfectly identical.

Why This Destroys Eve's Information: Eve's partial information (e.g., "I know that bits 1 through 8 have an even parity") is about the *input* to the blender. Because of the avalanche effect, this knowledge is completely useless for predicting the *output* of the blender. The relationship between the input and the output is so chaotic that her partial clues are effectively randomized into oblivion. She is left with a key that is shorter, but from her perspective, perfectly random and secret.

4.4 The Quantum Bit Error Rate (QBER): A Dual-Purpose Metric

The QBER is the most critical measurable parameter in a QKD system. It is calculated as the ratio of mismatched bits to the total number of bits in the publicly compared sample of the sifted key. It serves two distinct purposes:

1. **As a Channel Quality Metric:** It provides a direct measurement of the physical channel's noisiness. A higher QBER indicates a lower-quality transmission.
2. **As a Security Metric:** It is the primary indicator of eavesdropping. Any attempt by Eve to gain information about the transmitted qubits will inevitably disturb them, causing a measurable increase in the QBER.

4.5 The Security Threshold: A Deliberate Balance

The decision to accept or abort a key exchange is based on comparing the measured QBER to a pre-defined Security Threshold. The choice of this threshold (e.g., 8% or 10% in the simulation) is a critical trade-off between security and efficiency. As mathematically shown for the intercept-resend attack, Eve's actions are guaranteed to introduce a QBER of at least 25%. This provides a large, clear signature. The security threshold must be set comfortably below this value to ensure Eve cannot hide within the noise. This is the most crucial constraint. As will be detailed next, information theory proves that if the QBER exceeds a certain point, Eve could potentially have more information about the key than Bob does. At this point, no amount of post-processing can salvage a secret key. This hard limit for BB84 is approximately 11%. Therefore, any practical security threshold **must** be set below 11%. The choice of 8-10% in the simulation provides a safe operational margin below this theoretical "point of no return."

4.6 Quantifying Information Leakage: Privacy Amplification and Shannon Entropy

A secure key can only be generated if, after the quantum transmission, Alice and Bob share more information with each other than Eve has managed to learn about Alice's key. The final post-processing step, Privacy Amplification, is a procedure for distilling this "information advantage" into a shorter, perfectly secret key. The size of this advantage determines the final key length.

- I(A:B): The mutual information between Alice and Bob's sifted keys.
- I(A:E): The mutual information between Alice's key and Eve's knowledge.
- **Condition for Security:** A secure key can be generated only if $I(A : B) > I(A : E)$.

Information theory quantifies these values using the Shannon binary entropy function, $H(Q)$. It measures the uncertainty (in bits) of a binary random variable with a probability of error Q.

$$H(Q) = -Q \cdot \log_2(Q) - (1 - Q) \cdot \log_2(1 - Q) \tag{4.1}$$

The security proofs for BB84 (most notably the Shor-Preskill proof) connect the measurable QBER (Q) to the information held by all parties:

- Alice and Bob's shared information is reduced by the uncertainty: $I(A : B) = 1 - H(Q)$.
- Eve's maximum possible information is bounded by the disturbance she creates: $I(A : E) \leq H(Q)$.

To generate a secure key, we must satisfy the security condition $I(A : B) > I(A : E)$. Assuming the worst-case for Eve ($I(A : E) = H(Q)$), the condition becomes:

$$1 - H(Q) > H(Q) \implies 1 > 2 \cdot H(Q) \implies H(Q) < 0.5 \tag{4.2}$$

This inequality states that security is only possible if the channel's entropy is less than 0.5. We can numerically solve the equation $H(Q) = 0.5$ to find the critical QBER value at which the information advantage disappears. The solution is:

$$Q \approx 0.1100 \tag{4.3}$$

This means that for any QBER greater than or equal to 11%, Eve could know as much or more than Bob, and no secret key can be extracted.

CHAPTER 5

SIMULATION METHODOLOGY AND IMPLEMENTATION

5.1 Simulation Environment and Libraries

The entire simulation and analysis framework was developed in the Python 3 programming language, chosen for its extensive scientific computing ecosystem. The following open-source libraries were instrumental:

- **Qiskit (v0.x) & Qiskit-Aer:** The primary framework for quantum computation. Qiskit was used to construct the quantum circuits representing qubit preparation and measurement. The Qiskit-Aer backend provided the high-performance simulator capable of executing these circuits under the influence of complex, custom-defined noise models, including the use of density matrix simulations required for accurate noise modeling.
- **Pandas:** Utilized as the core data management tool. All results from individual simulation runs were collected into a Pandas DataFrame, which allowed for efficient aggregation, analysis, and subsequent export to a CSV file for persistence and reproducibility.
- **NumPy:** Served as the numerical engine for all mathematical operations. This included generating parameter ranges, performing statistical calculations, and critically, implementing the least-squares polynomial fitting for generating the 3D surfaces of best fit from the raw simulation data.
- **Matplotlib & Seaborn:** The primary visualization libraries. Matplotlib's `mpl_toolkits.mplot3d` was essential for rendering the 3D plots, while Seaborn was used to enhance the aesthetics and statistical clarity of all 2D graphs.

5.2 Architectural Overview

The project code is structured into four logical parts, reflecting the distinct stages of a scientific simulation:

1. **Simulation Components:** A collection of functions, each modeling a specific physical action or entity within the BB84 protocol (e.g., `alice_encodes`, `bob_measures`, `create_noise_model`).

2. **Core Simulation Engine (run_single_simulation):** A central function that orchestrates the components to execute one complete end-to-end run of the protocol for a given set of parameters.

3. **Experiment Orchestrator (run_experiment):** A high-level manager that systematically iterates through all specified parameter combinations (distance, noise, Eve's presence), calling the core engine for each and collecting the results.

4. **Data Analysis and Visualization:** A final script that loads the complete dataset and generates the analytical plots and surfaces.

5.3 The Noise Model Engine (create_noise_model)

This function is the heart of the physical channel simulation. Upon receiving a dictionary of parameters (`noise_type`, `error_rate`, `distance_km`), it constructs a Qiskit `NoiseModel` object through the following steps:

1. **Distance-Dependent Loss:** It first calculates the probability of photon loss due to distance. The Beer-Lambert Law is applied using a standard telecom fiber attenuation constant of 0.2 dB/km. The resulting loss probability is used to create an `amplitude_damping_error`. This error is applied to the `id` (identity) gate, realistically modeling the effect of the qubit simply traveling through the fiber over time.

2. **Intrinsic Channel Noise:** It then models the inherent noisiness of the channel. Based on the `noise_type` parameter, it creates the corresponding Qiskit error object (`depolarizing_error`, `phase_damping_error`, etc.).

3. **Combined Model Logic:** For the combined noise model, the intrinsic `error_rate` is divided by three, and one of each of the three primary error types is instantiated. These individual quantum errors are then composed into a single, more complex error channel using `err_dep.compose(err_amp).compose(err_phase)`.

4. **Model Finalization:** The intrinsic noise error is applied to the active quantum gates (x, h), modeling imperfections that occur during state manipulation. The function returns the completed `NoiseModel` object, ready to be passed to the Qiskit-Aer simulator.

5.4 The Simulation Orchestrator (run_experiment)

To ensure a comprehensive exploration of the parameter space, the `run_experiment` function uses a series of nested loops. The outermost loop iterates through the `noise_types`, followed by loops for `noise_rates`, distances, and finally, the binary choice of Eve's presence ([`True`, `False`]). The innermost loop (`runs_per_setting`) repeats the simulation for each unique

configuration to allow for statistical averaging and reduce the impact of probabilistic quantum outcomes. Before each call to `run_single_simulation`, a parameter dictionary is dynamically assembled, ensuring each run is executed with the correct settings. Progress is reported to the console periodically.

5.5 Data Aggregation and Surface Fitting

Once the raw simulation data is loaded from the CSV file, it is aggregated using the Pandas `groupby()` method. This groups all runs by their unique parameter set and computes the mean value for key metrics like `qber` and `final_key_length`.

For the 3D visualizations, a 2nd-order polynomial surface of the form:

$$z = a + bx + cy + dxy + ex^2 + fy^2 \tag{5.1}$$

was fitted to the aggregated data points. Here, x is distance, y is the noise rate, and z is the mean QBER. The coefficients (a through f) were solved for using NumPy's linear least-squares function, `np.linalg.lstsq`. This approach allows for the visualization of a smooth, continuous trend from the discrete simulation data points, making the underlying relationships much clearer.

CHAPTER 6

RESULTS: IN-DEPTH ANALYSIS AND INTERPRETATION

The results of the simulation are presented in a series of visualizations designed to explore the multi-dimensional parameter space.

6.1 Analysis of 3D QBER Surfaces (Individual Noise Models)

6.1.1 Observations

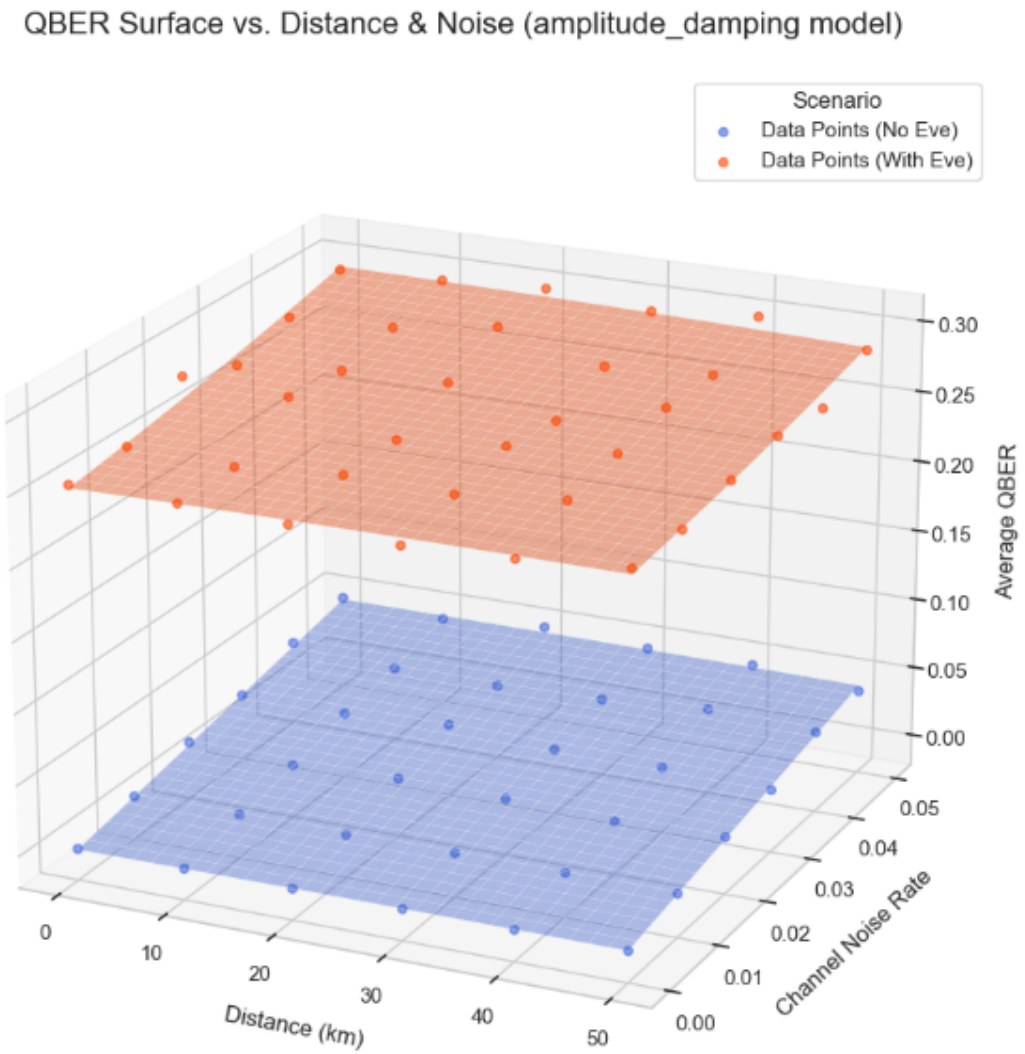

Figure 6.1: *QBER Surface for Depolarizing Noise Model, showing the clear separation between the attacked (upper) and unattacked (lower) channels.*

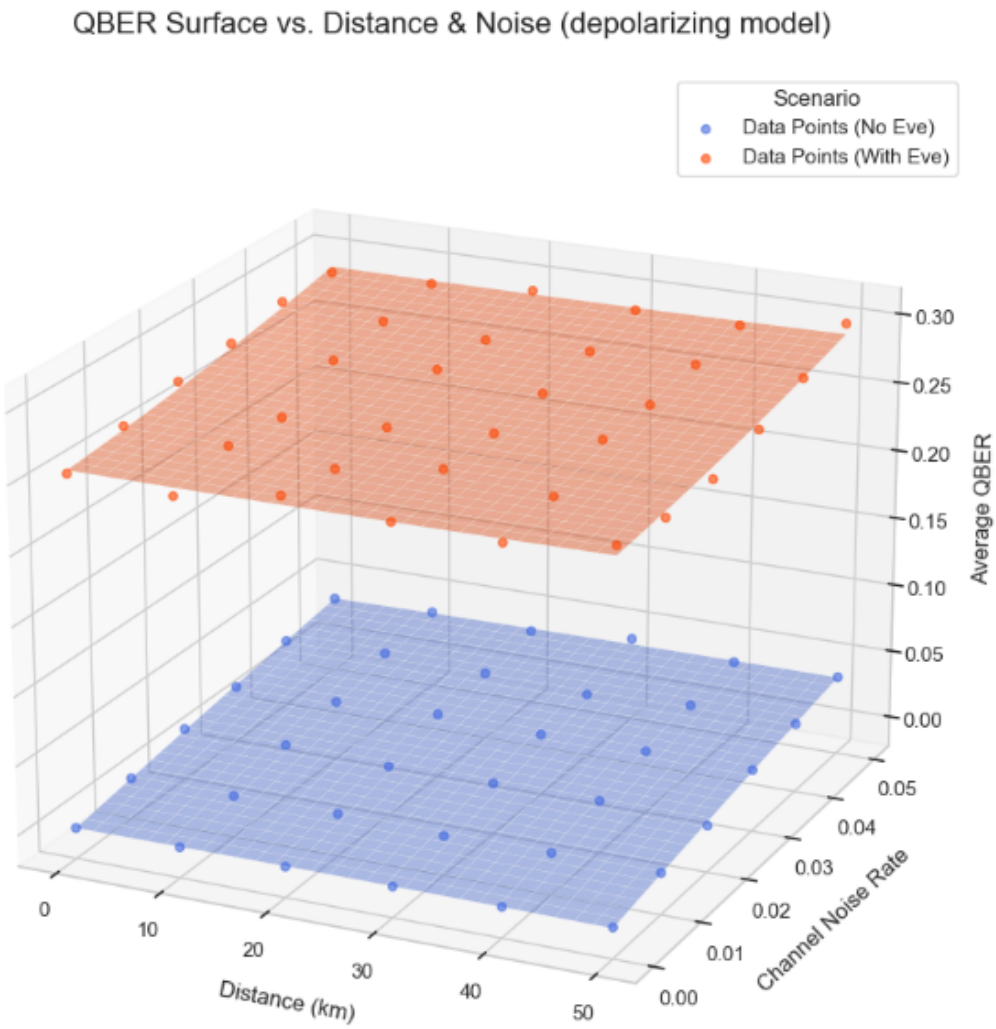

Figure 6.2: *QBER Surface for Amplitude Damping Noise Model. The behavior is qualitatively similar to the depolarizing model.*

The 3D surfaces for the individual depolarizing and amplitude damping noise models show qualitatively similar behavior. In both cases, two distinct surfaces are formed:

- **The "No Eve" Surface (Lower):** This surface starts near a QBER of 0 at zero distance and zero noise. It exhibits a positive gradient along both the distance and noise rate axes, forming a gently rising, slightly curved plane.
- **The "With Eve" Surface (Upper):** This surface is located significantly higher, originating at a QBER of approximately 0.25 even at zero distance and noise. It also shows a positive gradient, indicating that channel noise and distance effects are additive to the error already introduced by Eve.

6.1.2 Interpretation

These plots provide a powerful validation of BB84's security model. The significant and persistent vertical separation between the two surfaces is the "detection window." The lower surface represents the expected baseline error rate of the physical channel. The upper surface represents the channel *under attack*. The fact that these two surfaces do not intersect at any point in the simulated parameter space demonstrates that the QBER induced by an intercept-resend attack is fundamentally distinguishable from that induced by natural channel decoherence. An operator of a QKD system can measure the QBER and unambiguously determine which of the two regimes they are operating in.

6.2 Analysis of the 3D QBER Surface (Combined "Real-World" Model)

6.2.1 Observations

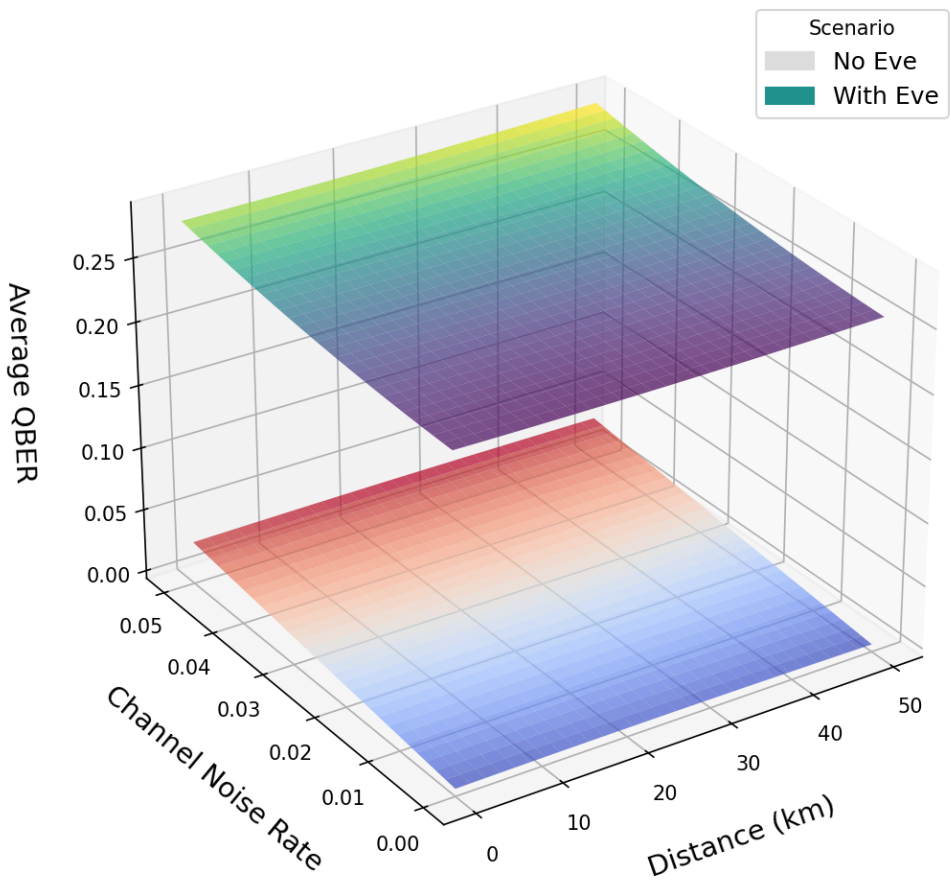

Figure 6.3: *QBER Surface for the Combined Noise Model (View 1).*

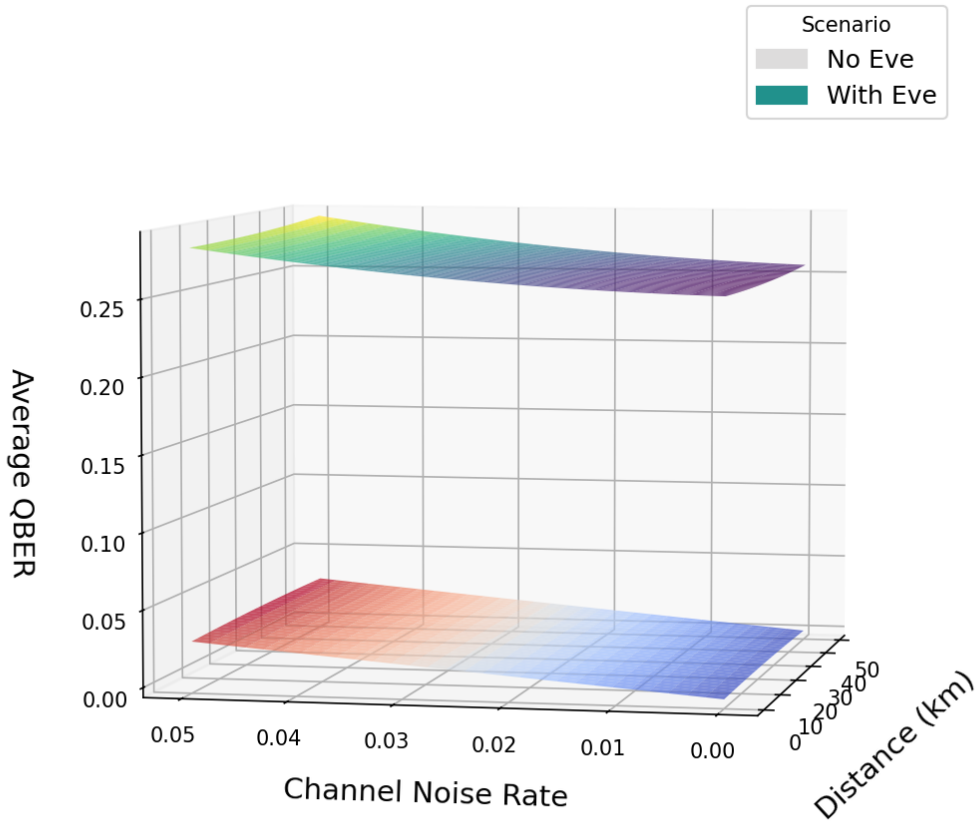

Figure 6.4: *QBER Surface for the Combined Noise Model (View 2), highlighting the gradient along the distance axis.*

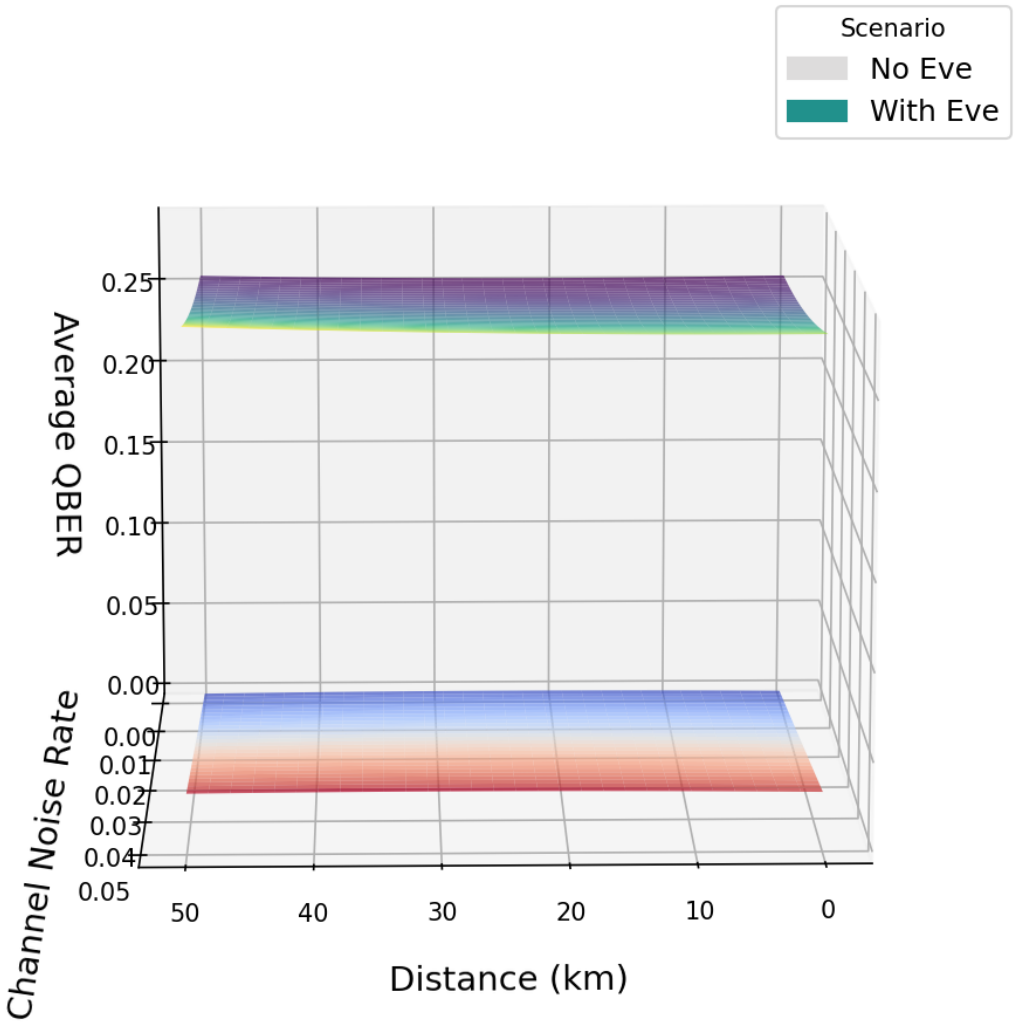

Figure 6.5: *QBER Surface for the Combined Noise Model (View 3), highlighting the gradient along the noise rate axis.*

This plot, representing the most realistic channel model, synthesizes the effects of all noise types. The observed behavior is consistent with the individual models: two clearly separated surfaces are present. The gradient of the lower ("No Eve") surface appears slightly steeper than in the individual models, particularly along the distance axis.

6.2.2 Interpretation

This result is critical, as it confirms that the security guarantee is not an artifact of using a simplified, single-type noise model. Even when a complex combination of physical noise processes is simulated, the QBER signature of an eavesdropper remains statistically unique. The steeper gradient of the "No Eve" surface is also expected; in the combined model, the QBER is a sum of errors from depolarizing, phase damping, and the ever-present amplitude damping from distance, leading to a faster accumulation of errors compared to any single model alone.

6.3 Analysis of Secure Key Rate vs. Distance

6.3.1 Observations

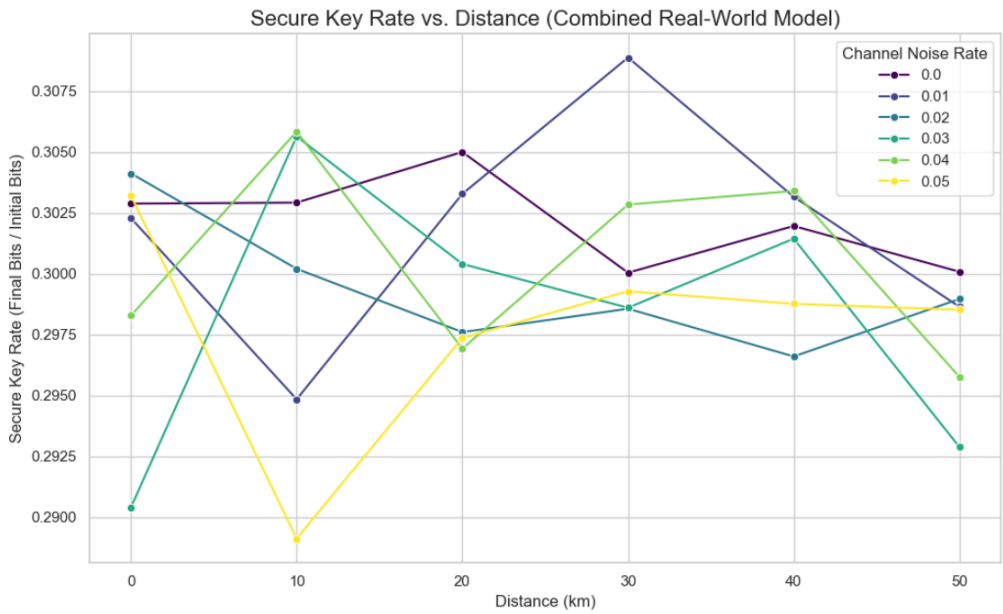

Figure 6.6: *Secure Key Rate (SKR) as a Function of Distance for different levels of intrinsic channel noise.*

This 2D plot shows the Secure Key Rate (efficiency) as a function of distance for the combined noise model in the absence of an eavesdropper. Each line represents a different level of intrinsic channel noise. Two trends are immediately apparent:

1. All lines exhibit a strong, non-linear decay with increasing distance.
2. Higher intrinsic noise rates (e.g., 5%) result in a lower SKR at all distances and cause the key rate to drop to zero at a shorter distance compared to a low-noise channel (e.g., 1%).

6.3.2 Interpretation

This graph clearly illustrates the practical performance limitations of fiber-based BB84. The sharp drop in SKR is a direct consequence of the Beer-Lambert Law: the exponential decay in photon survival probability means that the number of bits available for the sifted key plummets over longer distances. The second trend shows the interplay between distance-based loss and intrinsic noise. As both increase, the total QBER eventually crosses the pre-defined 10% security threshold, causing the protocol to abort. A noisier channel crosses this threshold at a shorter distance, thus explaining why the higher-noise lines terminate earlier. This plot is essential for engineering considerations, as it defines the maximum viable range of a QKD link for a given channel quality.

6.4 Analysis of 2D QBER Slices from the Combined Model

6.4.1 QBER vs. Channel Noise at a Fixed Distance

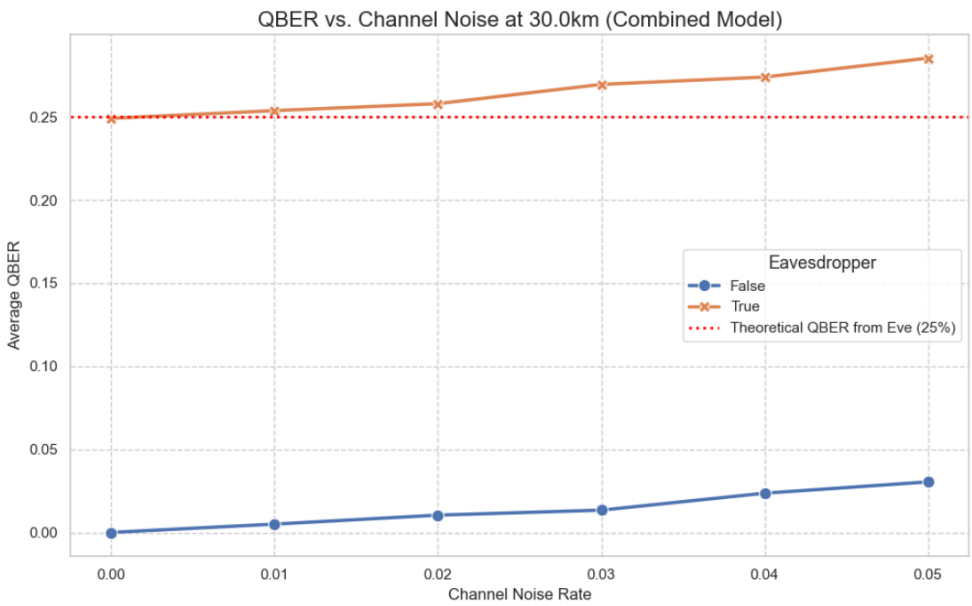

Figure 6.7: *QBER vs. Intrinsic Noise Rate at a Fixed Distance of 30 km.*

This plot shows a cross-section of the 3D data at a constant distance of 30 km. The "No Eve" line starts at a non-zero QBER (due to the 20 km of photon loss) and rises linearly with increasing intrinsic channel noise. The "With Eve" line starts at a much higher QBER and also rises with a similar slope. The gap between them remains large and nearly constant. This confirms that even in a channel with significant baseline loss, the additional QBER from an attack is easily detectable.

CHAPTER 7

DISCUSSION

7.1 Synthesis of Primary Findings

The collective results of this simulation lead to two primary, interwoven conclusions. First, the security of the BB84 protocol against an intercept-resend attack is demonstrably robust. The QBER provides a reliable and unambiguous metric for intrusion detection, a conclusion that holds true even within a complex, multi-faceted noise environment and across significant transmission distances. Second, the practical efficiency of the protocol, quantified by the Secure Key Rate, is fundamentally constrained by the physical properties of the quantum channel. The exponential signal attenuation described by the Beer-Lambert Law is the dominant factor limiting the maximum achievable distance of a point-to-point QKD link. Intrinsic channel noise further exacerbates this limitation, reducing both the key rate and the maximum range. This establishes a clear trade-off space for QKD system engineers: achieving security over longer distances necessitates higher-quality, lower-loss, and lower-noise components.

7.2 Limitations of the Simulation Model

While comprehensive, the simulation operates under several simplifying assumptions:

- **Omission of Post-Processing Costs:** The calculated Secure Key Rate does not account for the reduction in key length due to the information leakage during Error Correction or the final hashing of Privacy Amplification. The reported SKR is therefore an optimistic upper bound on the true, usable key rate.
- **Idealized Hardware:** The model assumes perfect state preparation and measurement capabilities. Real-world devices suffer from imperfections like detector "dark counts" (registering a photon when none is present) and state preparation fidelity errors, which would contribute to a higher baseline QBER.
- **Singular Attack Vector:** The simulation only models the canonical intercept-resend attack. While this is the most illustrative attack for demonstrating the QBER principle, more advanced attacks (e.g., Photon Number Splitting attacks, which exploit imperfect

single-photon sources) would require different security considerations and countermeasures not modeled here.

CHAPTER 8

THE PNS VULNERABILITY: FROM IDEAL TO REAL-WORLD QKD

To fully appreciate the solution that Decoy State QKD provides, we must first understand the severity of the problem it solves. The vulnerability arises from a critical difference between the theoretical BB84 protocol and its practical implementations.

8.1 The Imperfection of Real-World Sources

The core security proofs of BB84 rely on the assumption that Alice sends exactly one photon per pulse. However, creating a perfect, on-demand single-photon source is technologically challenging. Practical QKD systems therefore use a readily available alternative: a standard laser pulse that is heavily weakened or attenuated.

The photon count in a laser pulse is not fixed; it follows a statistical pattern known as a **Poisson distribution**. The probability, p_n, of a pulse containing exactly n photons is given by:

$$p_n(\mu) = \frac{e^{-\mu}\mu^n}{n!} \tag{8.1}$$

where μ is the average photon number per pulse, a parameter Alice can control. When Alice sets her laser to an average intensity of, say, $\mu = 0.5$ photons per pulse, the actual photon count for any given pulse will vary:

- Some pulses will have 0 photons (a vacuum).
- Some will have 1 photon (the ideal state for QKD).
- Some will have 2 or more photons (the source of the vulnerability).

An eavesdropper, Eve, can build a device to perform a Quantum Non-Demolition (QND) measurement. This special technique allows her to count the number of photons in a pulse without disturbing their quantum state (e.g., their polarization).

8.2 Eve's Attack Strategy: The Photon-Number-Splitting (PNS) Attack

Armed with a QND device, Eve can execute a devastatingly effective strategy:

1. **Intercept Everything:** Eve sits on the quantum channel and intercepts every pulse Alice sends to Bob.
2. **Count the Photons:** For each intercepted pulse, she uses her QND device to count the photons.
   - **If it's a 1-photon pulse:** Eve is in a bind. She cannot split the photon. If she measures it, she will disturb it and introduce a high error rate. If she blocks it, she will increase the channel loss. Both actions are detectable. Therefore, her optimal strategy is to block all single-photon pulses.
   - **If it's a 2+ photon pulse:** This is her golden opportunity. She performs the Photon-Number-Splitting (PNS) attack. She carefully "splits off" one photon and stores it in her quantum memory, then forwards the remaining identical photon(s) to Bob through a lossless channel.
3. **Gain Information Invisibly:**
   - Bob receives the forwarded photon from the multi-photon pulse, measures it, and perceives a normal, low-loss transmission.
   - Later, Alice and Bob publicly discuss their bases to create their sifted key. Eve listens to this discussion.
   - For every bit where Bob's basis matched Alice's, Eve now knows the correct basis to measure her stored photon. She measures her copy and learns the value of that key bit with 100% accuracy.
   - Crucially, because Eve only forwarded perfect copies from multi-photon pulses and blocked the single-photon pulses that would have revealed her tampering, this attack introduces **zero errors**. The Quantum Bit Error Rate (QBER) that Alice and Bob calculate remains clean, and they falsely believe their generated key is secure.

This attack highlights the severe danger of imperfect sources and necessitates a more advanced protocol.

CHAPTER 9

THE DECOY STATE COUNTERMEASURE

The genius of Decoy State QKD is that it forces Eve to reveal her selective blocking and forwarding strategy by setting a statistical trap. Alice achieves this by randomly and secretly modulating the intensity of her laser pulses before sending them.

9.1 Introducing Decoy States

In addition to her normal signal states, Alice prepares two other types of states, distinguished only by their average photon number (μ):

- **Signal State (intensity μ):** This is the "normal" intensity pulse (e.g., $\mu \approx 0.5$). It contains a mix of vacuum, single-photon, and multi-photon components according to the Poisson distribution. These pulses are primarily used to generate the final secret key.
- **Weak Decoy State (intensity ν):** A much weaker pulse, where $\nu \ll \mu$ (e.g., $\nu \approx 0.1$). Because of the Poisson statistics, this pulse is overwhelmingly likely to be either a vacuum or a single-photon pulse. The probability of it containing two or more photons is negligible. This state is the perfect probe to check if someone is tampering with single-photon pulses.
- **Vacuum State (intensity 0):** A pulse with zero photons. This is used as a control to accurately measure the background noise of Bob's detectors (his "dark count" rate).

9.2 The Protocol in Action

1. **Randomized Preparation:** For every signal she sends, Alice randomly chooses her bit (0/1), her basis (+/x), and her intensity (μ, ν, or 0). She keeps a detailed record of these choices but tells no one.
2. **Transmission and Measurement:** The stream of mixed-intensity pulses travels to Bob, potentially through Eve's interception device. Bob measures each incoming pulse with his own randomly chosen basis, as in standard BB84.
3. **The Public Discussion (The Reveal):** This is where the trap is sprung.

- First, Alice and Bob compare bases and create a raw sifted key.
- Next, for every single pulse that was sent, Alice publicly announces its intensity level. For example: ”Pulse #1 was a signal state, Pulse #2 was a weak decoy, Pulse #3 was a vacuum state...”

9.3 The Statistical ”Smoking Gun”: How Eve is Caught

Now that all received signals have been sorted into three bins (Signal, Weak Decoy, Vacuum), Alice and Bob can perform a comparative statistical analysis. They calculate two key parameters for each bin:

- **Yield (Y):** The fraction of sent pulses that were actually detected by Bob. For example, Y_μ is the yield for signal states and Y_ν is the yield for weak decoy states.
- **QBER (E):** The bit error rate for the detected pulses within that bin (e.g., E_μ, E_ν).

Eve, sitting in the middle, cannot tell the difference between a single-photon pulse that originated from a high-intensity signal state and one that came from a low-intensity weak decoy state. To her, a single photon is a single photon, and her strategy is to block all of them. This leads to a massive, detectable discrepancy in the statistics:

- **On a secure channel:** The yields should be proportional to their intensities. Y_ν will naturally be lower than Y_μ, but in a predictable way that corresponds to the channel's natural loss.
- **On a channel with Eve's PNS attack:** Eve's strategy of blocking all single-photon pulses has a dramatic effect.
  - The yield of the signal states, Y_μ, will be affected, but since she forwards the multi-photon components, it won't drop to zero.
  - The weak decoy states, however, are almost exclusively single-photon pulses. Since Eve blocks these, the measured yield, Y_ν, will be dramatically lower than what is physically expected.

9.4 Key Distillation and Security Guarantee

The observation that Y_ν has plummeted is the statistical smoking gun. By precisely comparing the yields (Y_μ, Y_ν, Y_0) and error rates (E_μ, E_ν, E_0), Alice and Bob can do more than just detect Eve. They can use these values to mathematically place a strict lower bound on the single-photon yield (Y_1) and an upper bound on the single-photon error rate (e_1).

These estimated parameters allow them to calculate the secure key rate ‘S‘ using the GLLP formula:

$$S \geq q\{-Q_\mu f(E_\mu)H_2(E_\mu) + Q_1[1 - H_2(e_1)]\} \tag{9.1}$$

where $Q_1 = Y_1 \cdot \mu e^{-\mu}$. This formula balances the raw secret information gained from single photons (the positive term) against the information leaked during error correction (the negative term). If the calculated risk is too high (i.e., the statistics deviate too far from the expected values, yielding a negative ‘S‘), they abort the protocol entirely.

CHAPTER 10

SIMULATION AND OPTIMIZATION FRAMEWORK

To investigate the practical performance of the decoy state protocol, we developed a sophisticated simulation and optimization framework in Python. This framework is designed to model the protocol under realistic physical constraints and to find the optimal operating parameters that maximize its efficiency.

10.1 Simulation Environment

The simulation is built using a suite of standard scientific Python libraries:

- **NumPy:** Serves as the numerical engine for all mathematical operations, including implementing the Poisson distribution and the secure key rate formula.
- **SciPy:** Its powerful ‘minimize‘ function is used as the optimization engine to search the complex parameter space.
- **Matplotlib:** Used for generating all 2D and 3D visualizations of the results.

10.2 Physical Model Parameters

The simulation core takes a set of fixed physical and system parameters that define the real-world hardware and channel conditions. For our analysis, we used the following standard values:

- Fiber Loss (α): 0.2 dB/km.
- Detector Efficiency (η_{det}): 15%.
- Dark Count Probability (p_{dark}): 1×10^{-7} per pulse.
- Intrinsic Error Rate (e_{det}): 1.5%.
- Error Correction Inefficiency (f_{ec}): 1.16.

10.3 The Core Simulation Engine

The central function, '$calculate_key_rate$', is a direct implementation of the GLLP security analysis. For a given distance and a set of protocol parameters (intensities and probabilities), it performs the following steps:

1. It calculates the overall channel transmittance, η.
2. It computes the theoretical "ground truth" yields (Y_n) and error rates (e_n) for n-photon states based on the physical parameters.
3. It calculates the observable gains (Q_μ) and QBERs (E_μ) for each decoy state by taking the Poisson-weighted average of the ground-truth values.
4. It implements a simplified but effective finite-key analysis. A statistical uncertainty penalty, proportional to $1/\sqrt{N_{detections}}$, is calculated based on the number of pulses sent and the detection rate. This penalty is used to derive pessimistic bounds: a lower bound on Y_1 and an upper bound on e_1.
5. Finally, it plugs these pessimistic, finite-key adjusted values into the GLLP secure key rate formula (Equation 4.1) to get the final key rate.

10.4 The Need for an Optimization Layer

The effectiveness of the decoy state protocol is critically dependent on the choice of its parameters (the intensities $\mu, \nu_1, ...$ and probabilities $P_\mu, P_{\nu_1}, ...$). A poor choice can lead to a drastically lower or even zero key rate. Furthermore, the optimal choice is not static; it changes with the channel distance and the total number of available pulses (the block size).

Manually finding the best parameters for every single scenario is impossible. Therefore, an automated optimization layer is essential. We use the '$scipy.optimize.minimize$' function to wrap our simulation core. For each scenario (a fixed distance and block size), the optimizer intelligently searches the multi-dimensional parameter space to find the specific set of intensities and probabilities that maximizes the output of the '$calculate_key_rate$' function.

This approach ensures that when we compare the performance of a 2-state, 3-state, and 4-state protocol, we are comparing the *best possible version* of each, making our final conclusions about their relative merits robust and unbiased.

CHAPTER 11

RESULTS AND DISCUSSION

The results of our investigation are presented in two stages. First, we analyze the output of a preliminary simulation using fixed, non-optimized protocol parameters. This provides a baseline understanding and highlights the limitations of a simplified approach. Second, we present the results from our full, optimization-enabled simulation, which reveals a much richer and more accurate picture of the protocol's performance landscape.

11.1 Preliminary Analysis with Fixed Parameters

Our initial simulation was run with a set of pre-defined, ”good enough” intensities and probabilities for the 2, 3, and 4-state protocols. The goal was to determine the optimal number of states under these fixed conditions.

11.1.1 Observations

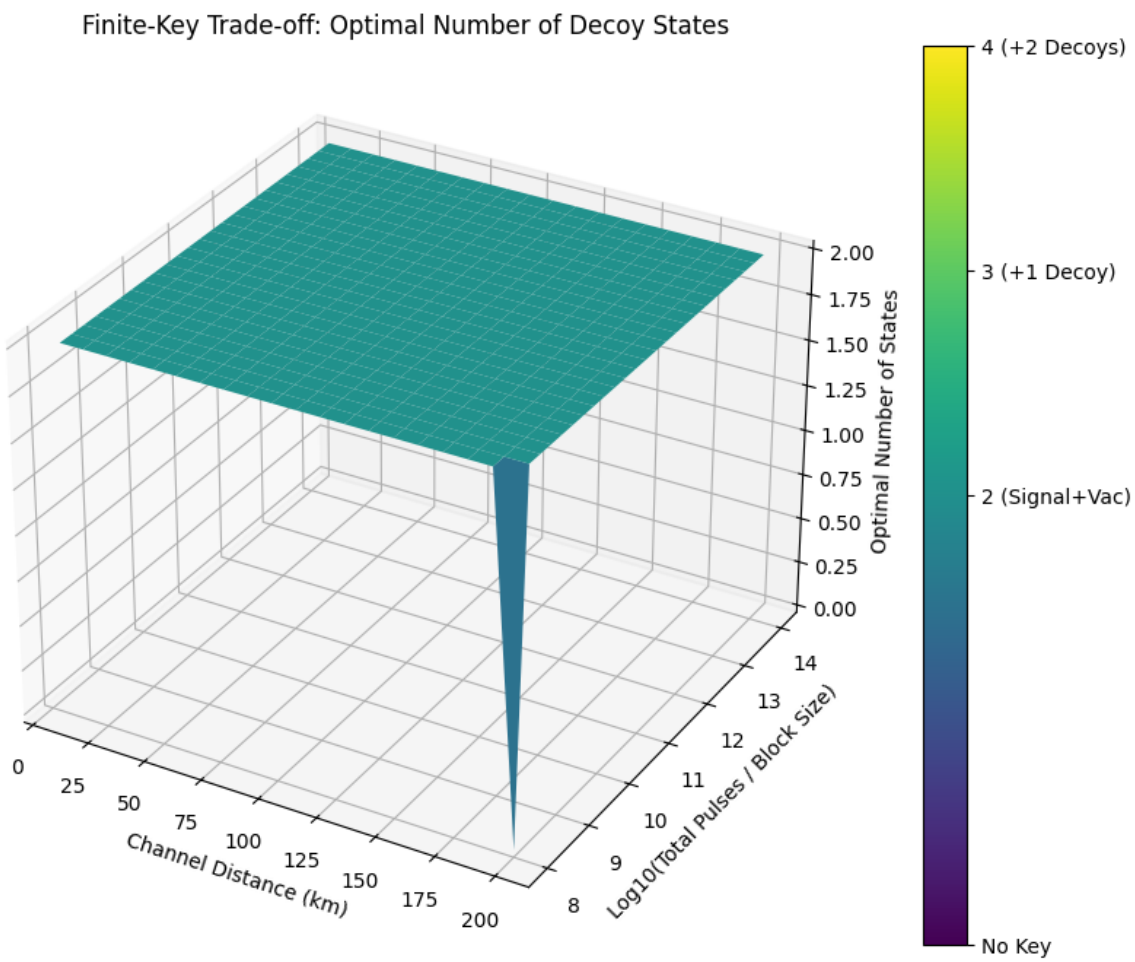

Figure 11.1: *Optimal number of decoy states determined using fixed, non-optimized protocol parameters. The result is a simple plateau.*

The result, shown in Figure 11.1, is a simple and unambiguous landscape.

- A single, dominant plateau colored green indicates that the **3-state protocol** (Signal + 1 Weak Decoy + Vacuum) is the optimal choice across all distances and block sizes where a secure key can be generated.
- A sharp ”cliff” is visible at long distances, where the surface drops to the purple ”No Key” region.

11.1.2 Initial Interpretation

This preliminary result suggests a simple rule: the 3-state protocol offers the best balance. The 2-state protocol is likely too simplistic to provide adequate security bounds, while the 4-state protocol suffers from a statistical penalty for splitting its pulses among too many bins, making

it less efficient. While compelling, this conclusion is potentially biased by the fixed parameters chosen for the simulation.

11.2 Robust Analysis with Full Parameter Optimization

To remove the bias of the preliminary analysis, a second, more computationally intensive simulation was performed. In this run, for every single point on the grid, an optimization algorithm was used to find the best possible intensities and probabilities for each of the 2, 3, and 4-state protocols before they were compared.

11.2.1 Observations

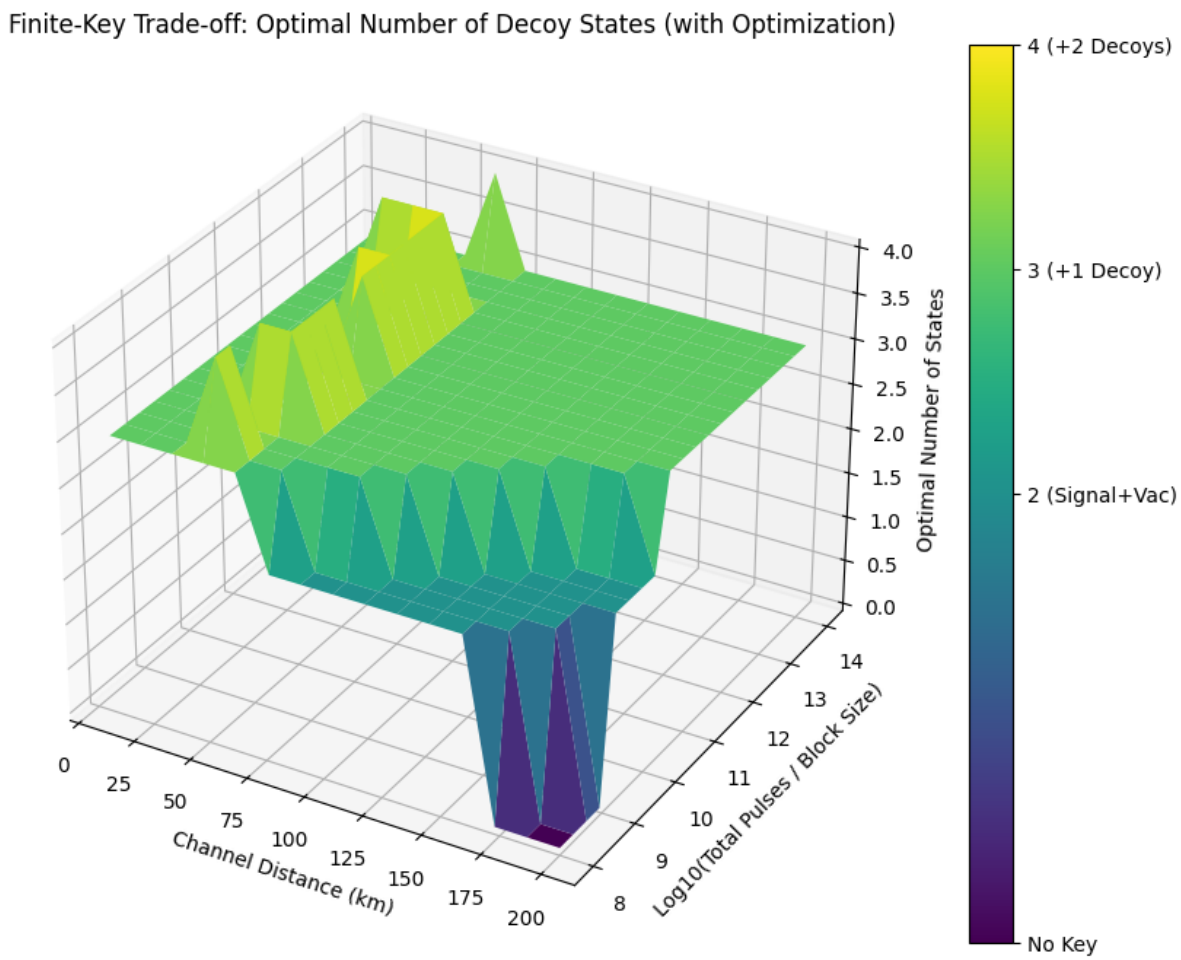

Figure 11.2: *Optimal number of decoy states determined using a full optimization of protocol parameters at each data point. This reveals a complex, multi-zoned "strategy map."*

The result of the full optimization, shown in Figure 11.2, is dramatically different and reveals a complex, multi-zoned landscape.

- The simple plateau is gone, replaced by distinct regions where different protocols are optimal.

- A significant **yellow region (4-state protocol)** appears at large block sizes and intermediate-to-long distances.
- The **green region (3-state protocol)** remains dominant but is now clearly bounded.
- Small **blue valleys (2-state protocol)** emerge at very small block sizes.

11.2.2 Interpretation: The Zones of Optimality

This ”strategy map” is a direct visualization of the finite-key trade-off.

Zone 1 (Data-Starved): 2-State Protocol. At very small block sizes ($N_{total} < 10^{10}$), there is not enough data to get reliable statistics for multiple decoy states. The optimizer finds that the simpler 2-state protocol, by concentrating all its pulses, can eke out a tiny, non-zero key rate while the more complex protocols are overwhelmed by statistical uncertainty.

Zone 2 (Practical Workhorse): 3-State Protocol. This protocol is optimal for the largest operational area, covering most practical block sizes and distances. It represents the best balance between providing a robust security estimate (unlike the 2-state protocol) and not paying too high a price in statistical uncertainty (unlike the 4-state protocol in this region).

Zone 3 (Data-Rich, Precision-Critical): 4-State Protocol. This is the most significant finding. When the block size is enormous ($N_{total} > 10^{12}$), the statistical cost of adding a fourth state becomes negligible. In this ”data-rich” environment, the superior precision of the 4-state model, which uses a second weak decoy to get a tighter bound on the channel’s properties, provides a marginal but decisive advantage, resulting in a higher key rate. This is especially true at longer distances where the signal is weak and precision is critical.

11.3 Deviation and Final Inferences

The stark difference between Figure 11.1 and Figure 11.2 is the primary finding of this work. It demonstrates that a simplified analysis can lead to incomplete or misleading conclusions. The ”normal” behavior suggested by the first plot—that the 3-state protocol is always best—was an artifact of the chosen fixed parameters, which were not optimal for the 4-state protocol in the specific regime where it could excel.

From the robust, optimized analysis, we deduce the following key inferences:

1. **There is no single ”best” protocol.** The optimal choice is a dynamic strategy that depends on the available resources (block size) and the physical challenge (distance).
2. **Finite-key effects are dominant.** The entire structure of the strategy map is a direct consequence of the statistical penalty associated with finite data, a factor that must be considered in any practical system design.

3. **Optimization is essential for fair comparison and peak performance.** To correctly evaluate QKD protocols and maximize their efficiency, it is crucial to optimize their operational parameters for the specific conditions in which they will be used. The results suggest that a sophisticated, real-world QKD system should be adaptive, capable of selecting and optimizing its protocol based on the link characteristics and session requirements.

CHAPTER 12

FINALCONCLUSION AND AVENUES FOR FUTURE RESEARCH

12.1 Conclusion

This simulation project successfully moved beyond a simplistic model of the BB84 protocol to a nuanced and physically-grounded analysis of its performance in a realistic, noisy environment. By implementing detailed models for distance-dependent amplitude damping, dephasing, and depolarizing noise, we have quantitatively demonstrated the protocol's operational limits. The results conclusively show that while the Secure Key Rate is fundamentally limited by the exponential signal loss with distance, the protocol's security is not compromised. The information-theoretic signature of an eavesdropper—a QBER of approximately 25%—remains clearly distinguishable from the baseline noise of the channel.

Furthermore, by addressing the critical Photon-Number-Splitting (PNS) vulnerability, we have developed a sophisticated optimization framework for the decoy state method. Our findings reveal that the optimal protocol configuration is not static but a dynamic choice dependent on the interplay between channel loss and the statistical limitations of a finite block size. The resulting "strategy map" provides a comprehensive, quantitative validation of BB84's security in non-ideal conditions and offers clear, data-driven guidelines for the engineering trade-offs required for its successful real-world implementation.

12.2 Avenues for Future Research

The successful development of the simulation and optimization framework in this project provides a robust foundation for several exciting avenues for future research, ranging from incremental model enhancements to a major new investigation into quantum network design.

12.2.1 Incremental Enhancements to the Simulation Model

The current simulation, while comprehensive, can be extended for even higher fidelity and broader applicability.

1. **Full Post-Processing Implementation:** The reported Secure Key Rate is an upper bound based on information-theoretic costs. Integrating a complete, classical post-processing stack (e.g., implementing the Cascade or LDPC protocol for error correction) would allow

for the calculation of the *true* final key rate after all information leakage is accounted for, providing a more accurate analysis of the protocol's end-to-end efficiency.

2. **Advanced Hardware Modeling:** The noise model could be extended to include specific hardware imperfections beyond the channel itself. This includes detector "dark counts," timing jitter, state preparation fidelity errors, and efficiency mismatch between detectors. This would allow for a more granular analysis of how the quality and cost of individual components affect overall system performance.

3. **Comparative Protocol Analysis:** The simulation framework could be adapted to model and compare the performance of other QKD protocols (e.g., SARG04, or the more advanced Measurement-Device-Independent QKD) under the same realistic channel and optimization conditions. This would provide valuable insights into their relative strengths and weaknesses in different operational regimes.

12.2.2 Primary Future Direction: The Quantum Repeater Placement Problem

The most significant conclusion from our results is the fundamental performance limitation imposed by distance. The exponential decay of the Secure Key Rate (SKR) makes it impossible to establish a secure key over a single fiber link beyond a few hundred kilometers. To build a long-distance or global quantum network, **quantum repeaters** or trusted nodes are essential.

Our current work provides the perfect tool to tackle this next-level challenge.

The Problem: From a Single Link to a Network Chain

A quantum repeater is a station placed between Alice and Bob that can overcome channel loss, effectively "stitching together" shorter, high-performance links to create a long-distance secure channel. The critical engineering question is: given a long-haul fiber optic cable between two cities (e.g., 1000 km apart), where is the optimal place to put these repeaters? Placing too few means the individual links will be too long and have a near-zero key rate. Placing too many increases cost, latency, and potential points of failure.

Leveraging the Current Work as a Predictive Engine

The simulation and optimization framework developed in this project is not merely an analysis tool; it is a **predictive engine for channel performance**. It can be abstracted into a function, $S_{max}(d, N)$, that provides the maximum possible secure key rate for a link of distance d and block size N. This function is the fundamental building block for solving the repeater placement problem.

Formulating the New Research Problem

The future research objective is to use our predictive engine to solve the following optimization problem:

- **Given:**
  - The total distance, D_{total}, between the end users.
  - A performance function, $S_{max}(d, N)$, derived from our existing simulation.
  - A set of constraints, such as a minimum required end-to-end secure key rate, S_{min}, and a maximum budget for the number of repeaters.
- **Objective:** Find the number of repeaters, k, and their individual placements (defining link lengths $d_1, d_2, ..., d_{k+1}$ where $\sum d_i = D_{total}$) that **maximizes the end-to-end key rate** of the entire chain, subject to the given constraints.

This problem can be approached using graph theory algorithms (e.g., a variation of Dijkstra's algorithm for finding the "highest-throughput" path) and numerical optimization techniques. The solution would provide a concrete, data-driven answer to one of the most pressing challenges in the engineering of large-scale quantum networks. In essence, this future work represents a natural evolution from the analysis of a single quantum link to the strategic design of a global quantum network infrastructure.

CHAPTER A

SOURCE CODE

This appendix contains the Python source code for the simulations described in this report.

A.1: Simulation with Fixed Parameters

This script was used to generate the preliminary results shown in Chapter 6. It compares the performance of 2, 3, and 4-state decoy protocols using a fixed set of "good enough" intensities and probabilities.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import math

# --- Physical and System Parameters ---
P = {
    'alpha': 0.2,       # Fiber loss in dB/km
    'eta_det': 0.15,      # Detector efficiency
    'p_dark': 1e-7,       # Dark count probability per pulse
    'e_det': 0.015,       # Intrinsic error rate of the detection system
    'f_ec': 1.16          # Error correction inefficiency
}

# --- Helper Functions ---
def H2(x):
    """Binary Shannon entropy function."""
    if x == 0 or x == 1:
        return 0
    return -x * np.log2(x) - (1 - x) * np.log2(1 - x)

def poisson(mu, n):
    """Probability of n photons in a pulse with average mu."""
    return np.exp(-mu) * (mu**n) / math.factorial(n)

# --- The Main Simulation Function ---
def run_simulation(distance, N_total, num_states):
    """
```

```
    Calculates the secure key rate for a given set of conditions.

    num_states: 2 (Signal+Vac), 3 (Signal+Weak+Vac), or 4 (Signal+2Weak+Vac
    )
    """
    # 1. Channel Transmittance
    eta_channel = 10**(-P['alpha'] * distance / 10)
    eta = eta_channel * P['eta_det']

    # 2. Define Protocol Parameters (intensities and probabilities)
    # These are pre-optimized for general use, but could also be optimized.
    if num_states == 2:
        intensities = {'mu': 0.5, 'vac': 0}
        probabilities = {'mu': 0.95, 'vac': 0.05}
    elif num_states == 3:
        intensities = {'mu': 0.5, 'nu1': 0.1, 'vac': 0}
        probabilities = {'mu': 0.8, 'nu1': 0.1, 'vac': 0.1}
    elif num_states == 4:
        intensities = {'mu': 0.6, 'nu1': 0.2, 'nu2': 0.05, 'vac': 0}
        probabilities = {'mu': 0.7, 'nu1': 0.1, 'nu2': 0.1, 'vac': 0.1}
    else:
        return -1 # Invalid number of states

    # 3. Calculate "True" Yields and Error Rates (Infinite Statistics)
    Y = np.zeros(5) # Yields for n=0 to 4 photons
    e = np.zeros(5) # Error rates for n=0 to 4 photons

    Y[0] = P['p_dark']
    e[0] = 0.5
    for n in range(1, 5):
        eta_n = 1 - (1 - eta)**n
        Y[n] = eta_n + P['p_dark'] - eta_n * P['p_dark']
        e[n] = (eta_n * P['e_det'] + 0.5 * P['p_dark']) / Y[n]

    # 4. Calculate "Observed" Gains and QBERs for each state
    gains = {}
    qbers = {}
    num_pulses = {}
    num_detections = {}

    for name, mu_val in intensities.items():
        Q = 0
        QE = 0
        for n in range(5):
            p_n = poisson(mu_val, n)
            Q += Y[n] * p_n
            QE += e[n] * Y[n] * p_n
```

```
        gains[name] = Q
        qbers[name] = QE / Q if Q > 0 else 0

        # Calculate number of pulses and detections for this state
        num_pulses[name] = N_total * probabilities[name]
        num_detections[name] = num_pulses[name] * gains[name]

    # 5. Finite-Key Analysis (Simplified Model)
    # We estimate the uncertainty in our knowledge of Y1 and e1

    # We use the weakest non-vacuum decoy to estimate the properties of single photons
    weakest_decoy_name = f'nu{num_states - 2}' if num_states > 2 else 'nu1' if num_states == 3 else 'mu'

    # Uncertainty is inversely proportional to sqrt of detections
    # Add a small epsilon to avoid division by zero
    detections_for_stats = num_detections.get(weakest_decoy_name, 1) + 1e-9

    # Simplified statistical fluctuation term. A larger number means more pessimistic.
    # In a real proof, this comes from the security parameter epsilon.
    stat_fluctuation_factor = 5.0
    uncertainty = stat_fluctuation_factor / np.sqrt(detections_for_stats)

    # Create pessimistic estimates for Y1 and e1
    # For a real proof, we'd solve a system of linear programs. Here, we use a direct penalty.
    Y1_pessimistic = Y[1] * (1 - uncertainty) # Lower bound on Y1
    e1_pessimistic = e[1] + (1 - e[1]) * uncertainty # Upper bound on e1
    e1_pessimistic = min(e1_pessimistic, 0.5)

    # 6. Calculate Secure Key Rate
    Q_signal = gains['mu']
    E_signal = qbers['mu']

    # Term from single-photon contributions
    Q1 = Y1_pessimistic * poisson(intensities['mu'], 1)

    # If our pessimistic estimate of Q1 is negative or e1 is too high, key rate is 0
    if Q1 <= 0 or e1_pessimistic >= 0.5:
        return 0.0

    term_privacy_amp = Q1 * (1 - H2(e1_pessimistic))
    term_error_corr = Q_signal * P['f_ec'] * H2(E_signal)
```

```python

    # Protocol efficiency (q=0.5 for BB84 sifting)
    q = 0.5 * probabilities['mu'] # We only generate key from signal states

    key_rate = q * (term_privacy_amp - term_error_corr)

    return max(0, key_rate)

# --- Main Execution and Plotting ---
if __name__ == "__main__":
    distances = np.linspace(10, 200, 20)  # 10 to 200 km
    log_block_sizes = np.linspace(8, 14, 25) # 10^8 to 10^14 pulses
    block_sizes = 10**log_block_sizes

    X, Y = np.meshgrid(distances, block_sizes)
    Z = np.zeros_like(X)

    print("Running simulation... This may take a few minutes.")

    for i in range(len(block_sizes)):
        for j in range(len(distances)):
            dist = X[i, j]
            n_tot = Y[i, j]

            # Compare the key rate for 2, 3, and 4 states
            rates = [run_simulation(dist, n_tot, num_s) for num_s in [2, 3, 4]]

            # Find which protocol was optimal (2, 3, or 4 states)
            if max(rates) > 0:
                Z[i, j] = np.argmax(rates) + 2
            else:
                Z[i, j] = 0 # No key could be generated
        print(f"Progress: {i+1}/{len(block_sizes)} complete.")

    # Plotting the 3D surface
    fig = plt.figure(figsize=(12, 8))
    ax = fig.add_subplot(111, projection='3d')

    # Use log scale for the block size axis for better visualization
    surf = ax.plot_surface(X, np.log10(Y), Z, cmap='viridis', rstride=1, cstride=1, edgecolor='none', vmin=0, vmax=4)

    ax.set_xlabel('Channel Distance (km)')
    ax.set_ylabel('Log10(Total Pulses / Block Size)')
    ax.set_zlabel('Optimal Number of States')
    ax.set_title('Finite-Key Trade-off: Optimal Number of Decoy States')
```

```python

    # Create a color bar with discrete labels
    cbar = fig.colorbar(surf, ticks=[0, 2, 3, 4])
    cbar.ax.set_yticklabels(['No Key', '2 (Signal+Vac)', '3 (+1 Decoy)', '4
    (+2 Decoys)'])

    plt.show()
```

Listing A.1: *Python code for the fixed-parameter simulation.*

A.2: Simulation with Parameter Optimization

This script contains the full optimization framework. It uses the SciPy library to find the optimal intensities and probabilities for each protocol at every data point before comparing their performance.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from scipy.optimize import minimize
import warnings
import math

# Suppress runtime warnings from log2(0) etc.
warnings.filterwarnings("ignore", category=RuntimeWarning)

# --- Physical and System Parameters ---
P = {
    'alpha': 0.2,       # Fiber loss in dB/km
    'eta_det': 0.15,      # Detector efficiency
    'p_dark': 1e-7,       # Dark count probability per pulse
    'e_det': 0.015,       # Intrinsic error rate of the detection system
    'f_ec': 1.16          # Error correction inefficiency
}

# --- Helper Functions ---
def H2(x):
    if x <= 0 or x >= 1:
        return 0
    return -x * np.log2(x) - (1 - x) * np.log2(1 - x)

_poisson_cache = {}
def poisson(mu, n):
    if (mu, n) in _poisson_cache:
        return _poisson_cache[(mu, n)]
    if n > 20: # Factorial becomes too large, but probability is negligible
        return 0.0
    result = np.exp(-mu) * (mu**n) / math.factorial(n)
    _poisson_cache[(mu, n)] = result
    return result

# --- Core Key Rate Calculation Function ---
def calculate_key_rate(distance, N_total, intensities, probabilities,
    p_phys):
    eta_channel = 10**(-p_phys['alpha'] * distance / 10)
    eta = eta_channel * p_phys['eta_det']

```

```
    Y = np.zeros(5)
    e = np.zeros(5)
    Y[0] = p_phys['p_dark']
    e[0] = 0.5
    for n in range(1, 5):
        eta_n = 1 - (1 - eta)**n
        Y[n] = eta_n + p_phys['p_dark'] - eta_n * p_phys['p_dark']
        if Y[n] > 0:
            e[n] = (eta_n * p_phys['e_det'] + 0.5 * p_phys['p_dark']) / Y[n
]

    gains = {}
    qbers = {}
    num_detections = {}
    for name, mu_val in intensities.items():
        Q, QE = 0, 0
        for n in range(5):
            p_n = poisson(mu_val, n)
            Q += Y[n] * p_n
            QE += e[n] * Y[n] * p_n
        gains[name] = Q
        qbers[name] = QE / Q if Q > 0 else 0
        num_detections[name] = N_total * probabilities[name] * gains[name]

    # --- Simplified Finite-Key Decoy Analysis ---
    # This is a key part. We must pessimistically estimate Y1 and e1.
    # We use the weak decoy state for this. For 4-state, we use both.
    # For simplicity, we use the 3-state logic which is standard.
    if 'nu1' in intensities:
        Q_nu1 = gains['nu1']
        E_nu1 = qbers['nu1']
        nu1 = intensities['nu1']

        # A simple but effective lower bound for Y1
        Y1_lower = (Q_nu1 * np.exp(nu1) - Y[0] - (gains['mu'] * np.exp(
intensities['mu']) * (nu1**2 / intensities['mu']**2))) / nu1

        # Upper bound for e1
        e1_upper = (E_nu1 * Q_nu1 * np.exp(nu1) - e[0]*Y[0]) / (Y1_lower *
nu1) if Y1_lower > 0 else 0.5
    else: # For 2-state, we have no good estimator, so we must be very
pessimistic
        Y1_lower = Y[1] * 0.5 # Pessimistic guess
        e1_upper = e[1] * 1.5 # Pessimistic guess

    Y1_lower = max(0, Y1_lower)
    e1_upper = min(0.5, e1_upper)
```

```python

    # Statistical fluctuation penalty
    weakest_decoy_name = 'nu1' if 'nu1' in intensities else 'mu'
    detections_for_stats = num_detections.get(weakest_decoy_name, 1) + 1e-9
    stat_fluctuation_factor = 5.0
    uncertainty = stat_fluctuation_factor / np.sqrt(detections_for_stats)

    Y1_pessimistic = Y1_lower * (1 - uncertainty)
    e1_pessimistic = e1_upper + (1 - e1_upper) * uncertainty
    e1_pessimistic = min(e1_pessimistic, 0.5)

    Q_signal = gains['mu']
    E_signal = qbers['mu']
    Q1 = Y1_pessimistic * poisson(intensities['mu'], 1)

    if Q1 <= 0 or e1_pessimistic >= 0.5: return 0.0

    term_privacy_amp = Q1 * (1 - H2(e1_pessimistic))
    term_error_corr = Q_signal * p_phys['f_ec'] * H2(E_signal)

    q = 0.5 * probabilities['mu']
    key_rate = q * (term_privacy_amp - term_error_corr)
    return max(0, key_rate)

# --- Optimization Wrappers ---
def optimize_protocol(distance, N_total, num_states):

    def objective_2_state(x):
        mu, P_mu = x
        intensities = {'mu': mu, 'vac': 0}
        probabilities = {'mu': P_mu, 'vac': 1 - P_mu}
        return -calculate_key_rate(distance, N_total, intensities, probabilities, P)

    def objective_3_state(x):
        mu, nu1, P_mu, P_nu1 = x
        intensities = {'mu': mu, 'nu1': nu1, 'vac': 0}
        probabilities = {'mu': P_mu, 'nu1': P_nu1, 'vac': 1 - P_mu - P_nu1}
        return -calculate_key_rate(distance, N_total, intensities, probabilities, P)

    def objective_4_state(x):
        mu, nu1, nu2, P_mu, P_nu1, P_nu2 = x
        intensities = {'mu': mu, 'nu1': nu1, 'nu2': nu2, 'vac': 0}
        probabilities = {'mu': P_mu, 'nu1': P_nu1, 'nu2': P_nu2, 'vac': 1 - P_mu - P_nu1 - P_nu2}
```

```
        return -calculate_key_rate(distance, N_total, intensities, probabilities, P)

    if num_states == 2:
        x0 = [0.5, 0.9]
        bounds = [(0.01, 1), (0.01, 0.99)]
        cons = []
        obj = objective_2_state
    elif num_states == 3:
        x0 = [0.5, 0.1, 0.7, 0.15]
        bounds = [(0.01, 1), (0.001, 0.4), (0.01, 0.98), (0.01, 0.49)]
        cons = [{'type': 'ineq', 'fun': lambda x: x[0] - x[1]}, # mu > nu1
                {'type': 'ineq', 'fun': lambda x: 1 - x[2] - x[3]}] # P_sum <= 1
        obj = objective_3_state
    elif num_states == 4:
        x0 = [0.6, 0.2, 0.05, 0.6, 0.1, 0.1]
        bounds = [(0.01, 1), (0.001, 0.5), (0.0001, 0.2), (0.01, 0.97), (0.01, 0.48), (0.01, 0.48)]
        cons = [{'type': 'ineq', 'fun': lambda x: x[0] - x[1]},      # mu > nu1
                {'type': 'ineq', 'fun': lambda x: x[1] - x[2]},      # nu1 > nu2
                {'type': 'ineq', 'fun': lambda x: 1 - x[3] - x[4] - x[5]}] # P_sum <= 1
        obj = objective_4_state
    else: return -1

    result = minimize(obj, x0, method='SLSQP', bounds=bounds, constraints=cons, options={'maxiter': 50})
    return -result.fun if result.success else 0.0

# --- Main Execution and Plotting ---
if __name__ == "__main__":
    distances = np.linspace(10, 200, 15)
    log_block_sizes = np.linspace(8, 14, 20)
    block_sizes = 10**log_block_sizes

    X, Y = np.meshgrid(distances, block_sizes)
    Z = np.zeros_like(X)

    print("Running simulation with optimization... This will take several minutes.")

    for i in range(len(block_sizes)):
        for j in range(len(distances)):
            dist = X[i, j]
```

```
            n_tot = Y[i, j]

            print(f"Optimizing for D={dist:.1f} km, logN={np.log10(n_tot):.1f}...")
            rates = [optimize_protocol(dist, n_tot, num_s) for num_s in [2, 3, 4]]

            if max(rates) > 1e-9: # Use a small threshold
                Z[i, j] = np.argmax(rates) + 2
            else:
                Z[i, j] = 0

    fig = plt.figure(figsize=(12, 8))
    ax = fig.add_subplot(111, projection='3d')

    surf = ax.plot_surface(X, np.log10(Y), Z, cmap='viridis', rstride=1, cstride=1, edgecolor='none', vmin=0, vmax=4)
    ax.set_xlabel('Channel Distance (km)')
    ax.set_ylabel('Log10(Total Pulses / Block Size)')
    ax.set_zlabel('Optimal Number of States')
    ax.set_title('Finite-Key Trade-off: Optimal Number of Decoy States (with Optimization)')

    cbar = fig.colorbar(surf, ticks=[0, 2, 3, 4])
    cbar.ax.set_yticklabels(['No Key', '2 (Signal+Vac)', '3 (+1 Decoy)', '4 (+2 Decoys)'])

    plt.show()
```

Listing A.2: *Python code for the optimization framework.*

A.3: BB84 protocol simulation with Qiskit-based noise modeling and statistical QBER analysis.

This script contains the full demonestration of BB84 along with simulations including noise. It uses the Qiskit open source quantum framework to simulate and derive results.

```
import random
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from qiskit import QuantumCircuit, Aer
from qiskit_aer.noise import (
    NoiseModel,
    depolarizing_error,
    amplitude_damping_error,
    phase_damping_error,
    QuantumError
)
from mpl_toolkits.mplot3d import Axes3D

# ==============================
# PART 1: SIMULATION COMPONENTS
# ==============================

def alice_encodes(bits, bases):
    """Creates a list of QuantumCircuits based on Alice's bits and bases.
    """
    qubits = []
    for i in range(len(bits)):
        qc = QuantumCircuit(1, 1)
        if bits[i] == 1:
            qc.x(0)
        if bases[i] == 1:  # 0 for Z-basis (+), 1 for X-basis (x)
            qc.h(0)
        qubits.append(qc)
    return qubits

def create_noise_model(params):
    """Creates a Qiskit noise model based on the full parameter dictionary.
    """
    model_type = params.get("noise_type")
    error_rate = params.get("error_rate", 0)
    distance_km = params.get("distance_km", 0)

    # Fiber attenuation constant in dB/km. Standard telecom fiber is around
     0.2 dB/km.
```

```
    ATTENUATION_DB_PER_KM = 0.2

    if error_rate == 0 and distance_km == 0:
        return None

    noise_model = NoiseModel()

    # --- Calculate distance-based amplitude damping (photon loss) ---
    # Convert dB loss to a probability of transmission
    # Loss(dB) = 10 * log10(P_in / P_out) => P_out / P_in = 10^(-Loss(dB)
   /10)
    total_loss_db = ATTENUATION_DB_PER_KM * distance_km
    transmission_prob = 10**(-total_loss_db / 10)
    # The error is the probability of *loss*, which is 1 - transmission
    amplitude_damping_from_distance = 1 - transmission_prob

    # This error is always present if distance > 0
    if distance_km > 0:
        error_amp_dist = amplitude_damping_error(
   amplitude_damping_from_distance)
        noise_model.add_all_qubit_quantum_error(error_amp_dist, ['id']) #
   Apply during "idle" time in fiber

    # --- Add channel noise based on model type ---
    if error_rate > 0:
        if model_type == "depolarizing":
            error = depolarizing_error(error_rate, 1)
            noise_model.add_all_qubit_quantum_error(error, ['x', 'h'])
        elif model_type == "amplitude_damping":
            error = amplitude_damping_error(error_rate)
            noise_model.add_all_qubit_quantum_error(error, ['x', 'h'])
        elif model_type == "phase_damping":
            error = phase_damping_error(error_rate)
            noise_model.add_all_qubit_quantum_error(error, ['x', 'h'])
        elif model_type == "combined":
            # Apply all three types of noise with a third of the error rate
    each
            rate_part = error_rate / 3
            err_dep = depolarizing_error(rate_part, 1)
            err_amp = amplitude_damping_error(rate_part)
            err_phase = phase_damping_error(rate_part)

            # Compose the errors into a single QuantumError
            combined_error = err_dep.compose(err_amp).compose(err_phase)
            noise_model.add_all_qubit_quantum_error(combined_error, ['x', '
   h'])
        else:
```

```
            raise ValueError("Unsupported noise model type specified.")

    return noise_model

def bob_measures(qubits, bases, noise_model=None):
    """Measures a list of qubits in specified bases, optionally with noise.
    """
    measured_bits = []
    backend = Aer.get_backend('aer_simulator')

    for i, qc in enumerate(qubits):
        if bases[i] == 1:
            qc.h(0)
        qc.measure(0, 0)

        # When running with a noise model, disable the built-in ideal
    simulator optimizations
        run_options = {}
        if noise_model:
            run_options['method'] = 'density_matrix'

        result = backend.run(qc, shots=1, noise_model=noise_model, **
    run_options).result()
        counts = result.get_counts(qc)

        # Handle the case of a lost photon (no measurement outcome)
        if not counts:
            measured_bit = random.randint(0, 1) # Assign a random bit for a
     lost photon
        else:
            measured_bit = int(list(counts.keys())[0])

        measured_bits.append(measured_bit)
    return measured_bits

def eve_intercepts_and_resends(qubits, params):
    """Eve performs an intercept-resend attack."""
    num_qubits = len(qubits)
    eve_bases = [random.randint(0, 1) for _ in range(num_qubits)]

    # Eve's measurement is subject to the same channel noise
    noise_model = create_noise_model(params)
    eve_measured_bits = bob_measures(qubits, eve_bases, noise_model)

    # Eve prepares and sends new qubits based on her results
    eves_forwarded_qubits = alice_encodes(eve_measured_bits, eve_bases)

```

```
    return eves_forwarded_qubits

def sift_keys(a_bases, b_bases, a_bits, b_bits):
    sifted_a, sifted_b = [], []
    for i in range(len(a_bases)):
        if a_bases[i] == b_bases[i]:
            sifted_a.append(a_bits[i])
            sifted_b.append(b_bits[i])
    return sifted_a, sifted_b

def calculate_qber_and_final_key(key_a, key_b, sample_size):
    key_len = len(key_a)
    if key_len == 0:
        return 0.0, []

    sample_size = min(sample_size, key_len)
    if sample_size == 0:
        return 0.0, key_a

    sample_indices = random.sample(range(key_len), sample_size)

    errors = sum(1 for i in sample_indices if key_a[i] != key_b[i])
    qber = errors / sample_size

    final_key = [k for i, k in enumerate(key_a) if i not in sample_indices]
    return qber, final_key

# ============================
# PART 2: CORE SIMULATION ENGINE
# ============================

def run_single_simulation(params):
    """Executes one full BB84 simulation for a given set of parameters."""
    noise_model = create_noise_model(params)

    alice_bits = [random.randint(0, 1) for _ in range(params["NUM_BITS"])]
    alice_bases = [random.randint(0, 1) for _ in range(params["NUM_BITS"])]
    alice_qubits = alice_encodes(alice_bits, alice_bases)

    if params["eve_present"]:
        # Eve's attack happens over the noisy channel from Alice to Eve
        qubits_for_bob = eve_intercepts_and_resends(alice_qubits, params)
    else:
        qubits_for_bob = alice_qubits

    # The final transmission from Eve (or Alice) to Bob is over another
    noisy channel
```

```
    bob_bases = [random.randint(0, 1) for _ in range(params["NUM_BITS"])]
    bob_results = bob_measures(qubits_for_bob, bob_bases, noise_model)

    sifted_alice, sifted_bob = sift_keys(alice_bases, bob_bases, alice_bits, bob_results)

    qber, final_key = calculate_qber_and_final_key(sifted_alice, sifted_bob, params["qber_sample_size"])

    aborted = qber > params["security_threshold"] and len(sifted_alice) > params["qber_sample_size"]

    results = {
        "initial_bits": params["NUM_BITS"],
        "noise_type": params["noise_type"],
        "error_rate": params["error_rate"],
        "distance_km": params["distance_km"],
        "eve_present": params["eve_present"],
        "sifted_key_length": len(sifted_alice),
        "final_key_length": 0 if aborted else len(final_key),
        "qber": qber,
        "protocol_aborted": aborted
    }
    return results

# ================================
# PART 3: EXPERIMENT ORCHESTRATOR
# ================================

def run_experiment(simulation_parameters, output_filename="bb84_simulation_data.csv"):
    all_results = []

    noise_rates = simulation_parameters["noise_rates"]
    noise_types = simulation_parameters["noise_types"]
    distances = simulation_parameters["distances"]
    runs_per_setting = simulation_parameters["runs_per_setting"]
    base_params = simulation_parameters["base_params"]

    total_sims = len(noise_rates) * len(noise_types) * len(distances) * 2 * runs_per_setting
    print(f"Starting experiment with a total of {total_sims} simulations...")

    sim_count = 0
    for n_type in noise_types:
        for rate in noise_rates:
```

```
            for dist in distances:
                for eve in [True, False]:
                    for _ in range(runs_per_setting):
                        sim_count += 1
                        params = base_params.copy()
                        params["noise_type"] = n_type
                        params["error_rate"] = rate
                        params["distance_km"] = dist
                        params["eve_present"] = eve

                        result = run_single_simulation(params)
                        all_results.append(result)

                        if sim_count % 50 == 0 or sim_count == total_sims:
                            print(f"  ... completed {sim_count}/{total_sims} simulations")

    df = pd.DataFrame(all_results)
    df.to_csv(output_filename, index=False)
    print(f"\nExperiment complete. Data saved to {output_filename}")
    return df

# ====================================
# PART 4: DATA ANALYSIS AND VISUALIZATION
# ====================================

def analyze_and_plot_results(df):
    print("\n--- Generating Analysis Plots ---")
    sns.set_theme(style="whitegrid")

    # Aggregate data for plotting
    agg_df = df.groupby(["noise_type", "error_rate", "distance_km", "eve_present"]).agg(
        mean_qber=('qber', 'mean'),
        mean_final_key_len=('final_key_length', 'mean'),
        abort_rate=('protocol_aborted', 'mean')
    ).reset_index()

    # --- Plot 1 & 2: 3D QBER vs. Distance vs. Noise (Individual Noise Models) ---
    noise_models_individual = [n for n in df['noise_type'].unique() if n != 'combined']
    for noise_model in noise_models_individual:
        fig = plt.figure(figsize=(12, 8))
        ax = fig.add_subplot(111, projection='3d')

        for eve, marker in [(False, 'o'), (True, '^')]:
```

```
            subset = agg_df[(agg_df['noise_type'] == noise_model) & (agg_df['eve_present'] == eve)]
            if not subset.empty:
                ax.scatter(subset['distance_km'], subset['error_rate'], subset['mean_qber'],
                           marker=marker, label=f'Eve Present: {eve}')

        ax.set_xlabel('Distance (km)')
        ax.set_ylabel('Channel Noise Rate')
        ax.set_zlabel('Average QBER')
        ax.set_title(f'QBER vs. Distance & Noise ({noise_model} model)')
        ax.legend()
        plt.savefig(f"3d_qber_{noise_model}.png")
        plt.show()

    # --- Plot 3: 3D QBER vs. Distance vs. Noise (Combined Model) ---
    fig = plt.figure(figsize=(12, 8))
    ax = fig.add_subplot(111, projection='3d')
    for eve, marker in [(False, 'o'), (True, '^')]:
        subset = agg_df[(agg_df['noise_type'] == 'combined') & (agg_df['eve_present'] == eve)]
        if not subset.empty:
            ax.scatter(subset['distance_km'], subset['error_rate'], subset['mean_qber'],
                       marker=marker, label=f'Eve Present: {eve}')
    ax.set_xlabel('Distance (km)')
    ax.set_ylabel('Channel Noise Rate')
    ax.set_zlabel('Average QBER')
    ax.set_title('QBER vs. Distance & Noise (Combined Real-World Model)')
    ax.legend()
    plt.savefig("3d_qber_combined.png")
    plt.show()

    # --- Plot 4: Secure Key Rate vs. Noise (Combined Model) ---
    # We only care about key rate when Eve is NOT present.
    # We plot against distance, with noise as different lines.
    key_rate_df = agg_df[(agg_df["eve_present"] == False) & (agg_df["noise_type"] == 'combined')]
    key_rate_df['key_rate'] = key_rate_df['mean_final_key_len'] / df['initial_bits'].iloc[0]

    plt.figure(figsize=(12, 7))
    sns.lineplot(data=key_rate_df, x="distance_km", y="key_rate", hue="error_rate", marker='o',
                 palette="viridis")
    plt.title("Secure Key Rate vs. Distance (Combined Real-World Model)", fontsize=16)
```

```
plt.xlabel("Distance (km)", fontsize=12)
plt.ylabel("Secure Key Rate (Final Bits / Initial Bits)", fontsize=12)
plt.legend(title="Channel Noise Rate")
plt.grid(True)
plt.savefig("key_rate_vs_distance_combined.png")
plt.show()

# --- Plot 5 & 6: 2D QBER vs. Noise / Distance (Combined Model) ---
# Plotting QBER vs Noise at a fixed distance (e.g., the middle value)
fixed_dist = sorted(df['distance_km'].unique())[len(df['distance_km'].unique()) // 2]
qber_vs_noise_df = agg_df[(agg_df["noise_type"] == 'combined') & (agg_df["distance_km"] == fixed_dist)]

plt.figure(figsize=(12, 7))
sns.lineplot(data=qber_vs_noise_df, x="error_rate", y="mean_qber", hue="eve_present", style="eve_present", marker='o')
plt.axhline(y=0.25, color='red', linestyle='--', label='Theoretical QBER from Eve (25%)')
plt.title(f"QBER vs. Channel Noise at {fixed_dist}km (Combined Model)", fontsize=16)
plt.xlabel("Channel Noise Rate", fontsize=12)
plt.ylabel("Average QBER", fontsize=12)
plt.legend(title="Eavesdropper")
plt.grid(True)
plt.savefig("qber_vs_noise_combined.png")
plt.show()

# Plotting QBER vs Distance at a fixed noise rate (e.g., the middle value)
fixed_noise = sorted(df['error_rate'].unique())[len(df['error_rate'].unique()) // 2]
qber_vs_dist_df = agg_df[(agg_df["noise_type"] == 'combined') & (agg_df["error_rate"] == fixed_noise)]

plt.figure(figsize=(12, 7))
sns.lineplot(data=qber_vs_dist_df, x="distance_km", y="mean_qber", hue="eve_present", style="eve_present", marker='o')
plt.axhline(y=0.25, color='red', linestyle='--', label='Theoretical QBER from Eve (25%)')
plt.title(f"QBER vs. Distance at {fixed_noise*100:.1f}% Channel Noise (Combined Model)", fontsize=16)
plt.xlabel("Distance (km)", fontsize=12)
plt.ylabel("Average QBER", fontsize=12)
plt.legend(title="Eavesdropper")
plt.grid(True)
plt.savefig("qber_vs_distance_combined.png")
```

```python
    plt.show()


# ==================================================
# MAIN EXECUTION BLOCK
# ==================================================

if __name__ == "__main__":
    SIMULATION_PARAMETERS = {
        # Define ranges to sweep through
        "distances": np.linspace(0, 50, 6),  # 0 to 50 km in 10 km steps
        "noise_rates": np.linspace(0, 0.05, 6), # 0% to 5% intrinsic channel noise

        # Define the types of noise to model, including the new 'combined' model
        "noise_types": ["depolarizing", "amplitude_damping", "combined"],

        # Number of times to run for each unique setting
        "runs_per_setting": 100, # NOTE: Lowered for quicker execution. Increase for smoother graphs.

        # Fixed parameters for all simulations
        "base_params": {
            "NUM_BITS": 250,           # Lowered for quicker execution
            "qber_sample_size": 50,
            "security_threshold": 0.10 # Slightly increased to see key rates at higher noise/distance
        }
    }

    results_df = run_experiment(SIMULATION_PARAMETERS)
    analyze_and_plot_results(results_df)
```

Listing A.3: *End-to-end BB84 simulation framework incorporating realistic quantum noise, distance-dependent losses, and intercept–resend eavesdropping.*

A.4: Interactive 3D QBER analysis showing the impact of distance and channel noise on BB84 performance, comparing scenarios with and without an eavesdropper (Eve)

The plot visualizes how realistic channel noise and distance affect QBER in BB84, highlighting the clear distinction between normal operation and eavesdropping scenarios..

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.patches as mpatches # Import patches for the legend

def create_interactive_plot_from_csv(filename="bb84_simulation_data.csv"):
    """
    Loads data from the CSV and generates a fully interactive 3D plot with:
    - Smooth, shaded surfaces for 'With Eve' and 'No Eve' scenarios.
    - Interactive rotation and zoom.
    - Coordinate display on mouse click.
    """
    try:
        df = pd.read_csv(filename)
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found.")
        print("Please make sure the CSV file is in the same directory as this script.")
        return

    print("--- Preparing Interactive 3D Plot ---")

    # Aggregate the data to get mean QBER values
    agg_df = df.groupby(["noise_type", "error_rate", "distance_km", "eve_present"]).agg(
        mean_qber=('qber', 'mean')
    ).reset_index()

    # We will plot the "combined" noise model as it's the most realistic
    plot_data = agg_df[agg_df['noise_type'] == 'combined']

    # --- Set up the Figure and 3D Axis ---
    fig = plt.figure(figsize=(16, 12))
    ax = fig.add_subplot(111, projection='3d')
    fig.canvas.manager.set_window_title('Interactive 3D QBER Analysis')

    # --- Data for the two scenarios ---
    data_no_eve = plot_data[plot_data['eve_present'] == False]
    data_with_eve = plot_data[plot_data['eve_present'] == True]

```

```
    # This will store the last annotation to be removed on the next click
    last_annotation = None

    # --- Plotting Surfaces and Points ---
    for data, color_map, label in [(data_no_eve, 'coolwarm', 'No Eve'), (data_with_eve, 'viridis', 'With Eve')]:
        if data.empty:
            continue

        x_data = data['distance_km']
        y_data = data['error_rate']
        z_data = data['mean_qber']

        # Fit a 2nd-order polynomial surface
        A = np.c_[np.ones(len(x_data)), x_data, y_data, x_data*y_data, x_data**2, y_data**2]
        coeffs, _, _, _ = np.linalg.lstsq(A, z_data, rcond=None)

        # Create a regular grid for plotting the surface
        x_range = np.linspace(x_data.min(), x_data.max(), 30)
        y_range = np.linspace(y_data.min(), y_data.max(), 30)
        X, Y = np.meshgrid(x_range, y_range)

        # Evaluate the polynomial on the grid
        Z = (coeffs[0] + coeffs[1]*X + coeffs[2]*Y +
             coeffs[3]*X*Y + coeffs[4]*X**2 + coeffs[5]*Y**2)

        # Plot the smooth, shaded surface
        ax.plot_surface(X, Y, Z, cmap=color_map, alpha=0.7, rstride=1, cstride=1,
                        linewidth=0.1, antialiased=True)

    # Combine all data points for the click-handler
    all_points_3d = np.array(plot_data[['distance_km', 'error_rate', 'mean_qber']])

    # --- The Event Handler for Mouse Clicks ---
    def on_click(event):
        nonlocal last_annotation

        if event.inaxes != ax:
            return

        points_2d = ax.transData.transform(all_points_3d)[:, :2]
        mouse_pixel_coords = np.array([event.x, event.y])
        distances = np.sqrt(np.sum((points_2d - mouse_pixel_coords)**2, axis=1))
```

```
        closest_index = np.argmin(distances)

        if distances[closest_index] < 15: # Sensitivity for clicking
            closest_point_3d = all_points_3d[closest_index]

            if last_annotation:
                last_annotation.remove()

            text = (f" Dist: {closest_point_3d[0]:.1f} km\n"
                    f" Noise: {closest_point_3d[1]*100:.1f} %\n"
                    f" QBER: {closest_point_3d[2]:.3f}")

            last_annotation = ax.text(closest_point_3d[0], closest_point_3d[1], closest_point_3d[2],
                                      text,
                                      color='black',
                                      bbox=dict(boxstyle="round,pad=0.3", fc="yellow", ec="black", lw=1, alpha=0.9),
                                      ha='center')

            fig.canvas.draw_idle()

    # Connect the click event to our handler function
    fig.canvas.mpl_connect('button_press_event', on_click)

    # --- Final Plot Styling ---
    ax.set_xlabel('Distance (km)', fontsize=14, labelpad=15)
    ax.set_ylabel('Channel Noise Rate', fontsize=14, labelpad=15)
    ax.set_zlabel('Average QBER', fontsize=14, labelpad=15)
    ax.set_title('Interactive QBER Surface vs. Distance & Noise (Combined Model)', fontsize=18, pad=20)

    # --- Corrected Legend Creation ---
    # Create proxy artists (Rectangles) but DO NOT add them to the axes with add_patch.
    # We only use them as handles for the legend.
    no_eve_proxy = mpatches.Patch(color=plt.get_cmap('coolwarm')(0.5), label='No Eve')
    with_eve_proxy = mpatches.Patch(color=plt.get_cmap('viridis')(0.5), label='With Eve')
    ax.legend(handles=[no_eve_proxy, with_eve_proxy], title="Scenario", fontsize=12)

    ax.view_init(elev=25, azim=-120) # Set a nice initial viewing angle
    plt.tight_layout()
    plt.show()

```

```python
if __name__ == "__main__":
    create_interactive_plot_from_csv()
```

Listing A.4: *Interactive 3d surface plot for QKD analysis.*

REFERENCES

[1] Charles H Bennett and Gilles Brassard. "Quantum cryptography: Public key distribution and coin tossing". In: *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing*. 1984.

[2] Gilles Brassard and Louis Salvail. "Secret-key reconciliation by public discussion". In: *Advances in Cryptology—EUROCRYPT '93*. Springer, 1994, pp. 410–423.

[3] Imre Csiszár and János Körner. "Broadcast channels with confidential messages". In: *IEEE Transactions on Information Theory* 24.3 (1978), pp. 339–348.

[4] Artur K Ekert. "Quantum cryptography based on Bell's theorem". In: *Physical Review Letters* 67.6 (1991), pp. 661–663.

[5] Nicolas Gisin et al. "Quantum cryptography". In: *Reviews of Modern Physics* 74.1 (2002), pp. 145–195.

[6] Daniel Gottesman et al. "Security of quantum key distribution with imperfect devices". In: *Quantum Information and Computation* 4.5 (2004), pp. 325–360.

[7] Won-Young Hwang. "Decoy state quantum key distribution". In: *Physical review letters* 91.5 (2003), p. 057901.

[8] Gerd Keiser. *Optical Fiber Communications*. 4th. McGraw-Hill, 2003.

[9] Xiongfeng Ma et al. "Practical decoy state for quantum key distribution". In: *Physical Review A* 72.1 (2005), p. 012326.

[10] Michael A Nielsen and Isaac L Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[11] Renato Renner. "Security of Quantum Key Distribution". In: *International Journal of Quantum Information* 6.1 (2008), pp. 1–127.

[12] Peter W Shor and John Preskill. "Simple proof of security of the BB84 quantum key distribution protocol". In: *Physical review letters* 85.2 (2000), p. 441.

[13] Stephen Wiesner. "Conjugate Coding". In: *SIGACT News* 15.1 (1983), pp. 78–88.

CHAPTER B

PLAGIARISM REPORT

Rishi Sarkar

QKD-Rishi-CSB22005

Dissertation/Thesis/Project Work/Assignment

Document Details

Submission ID

trn:oid:::3618:124986012

Submission Date

Dec 22, 2025, 12:05 PM GMT+5:30

Download Date

Dec 22, 2025, 12:23 PM GMT+5:30

File Name

QKD-Rishi-CSB22005.pdf

File Size

2.2 MB

78 Pages

18,333 Words

96,904 Characters

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 14 words)

Match Groups

19 Not Cited or Quoted 3%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
0% Publications
3% Submitted works (Student Papers)

Match Groups

- **19** Not Cited or Quoted 3%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
0% Publications
3% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.slideshare.net	<1%
2	Student papers	Tezpur University - CN-173457 on 2020-07-11	<1%
3	Student papers	Tezpur University - CN-173457 on 2019-12-16	<1%
4	Student papers	Tezpur University - CN-173457 on 2024-08-29	<1%
5	Student papers	The University of Manchester on 2023-04-21	<1%
6	Internet	www.inderscience.com	<1%
7	Student papers	University of Malta on 2025-05-09	<1%
8	Internet	eitca.org	<1%
9	Student papers	Tezpur University - CN-173457 on 2020-06-17	<1%
10	Student papers	Technical University of Cluj-Napoca on 2025-07-19	<1%

11 Publication

Wolfram Helwig, Wolfgang Mauerer, Christine Silberhorn. "Multimode states in d... <1%

12 Student papers

Nanyang Technological University on 2025-04-14 <1%

13 Internet

kipdf.com <1%

14 Internet

www.ir.juit.ac.in:8080 <1%

15 Student papers

VIT University on 2025-04-08 <1%